

NAME: T. SAI VENKATA SIVA REDDY

REG NO:192465021

COURSE CODE:CSA5801

COURSE NAME:DEVSCOPS ENGINEERING

#

=====

CSA58 DevSecOps Engineering and Application Security

40 R PROGRAMS

Rule:

Static / workflow / design / modelling -> DiagrammeR

Dynamic / simulation / analysis / results -> dygraphs

#

=====

Install once:

install.packages("DiagrammeR")

install.packages("dygraphs")

install.packages("xts")

library(DiagrammeR)

library(dygraphs)

library(xts)

```
#  
=====
```

```
# EXPERIMENT 1
```

```
# Installation and Configuration of Git, GitHub, RStudio
```

```
# and Required DevSecOps Tools
```

```
#  
=====
```

```
library(DiagrammeR)
```

```
grViz("
```

```
digraph DevSecOpsSetup {
```

```
graph [  
  rankdir = TB,  
  bgcolor = white,  
  labelloc = t,  
  label = 'DevSecOps Tool Installation and Configuration',  
  fontsize = 20  
]
```

```
node [  
  shape = box,  
  style = 'rounded, filled',  
  fontname = Helvetica,
```

```
    fontsize = 14  
  ]
```

```
Git [  
  label = '1. Install Git',  
  fillcolor = lightblue  
]
```

```
GitHub [  
  label = '2. Create / Configure GitHub',  
  fillcolor = lightgreen  
]
```

```
RStudio [  
  label = '3. Install RStudio',  
  fillcolor = lightyellow  
]
```

```
R [  
  label = '4. Configure R Environment',  
  fillcolor = lightpink  
]
```

```
Tools [  
  label = '5. Configure DevSecOps Tools',  
  fillcolor = lightcyan  
]
```

```
]
```

```
Verify [
```

```
  label = '6. Verify Installation',
```

```
  shape = diamond,
```

```
  fillcolor = lightgray
```

```
]
```

```
Ready [
```

```
  label = 'DevSecOps Environment Ready',
```

```
  shape = oval,
```

```
  fillcolor = lightgreen
```

```
]
```

```
Git -> GitHub
```

```
GitHub -> RStudio
```

```
RStudio -> R
```

```
R -> Tools
```

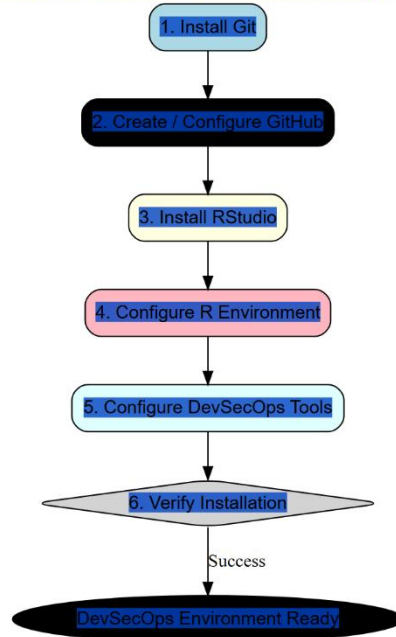
```
Tools -> Verify
```

```
Verify -> Ready [label = 'Success']
```

```
}
```

```
")
```

DevSecOps Tool Installation and Configuration



```

#
=====
=====

# EXPERIMENT 2

# Visualization of the DevSecOps Lifecycle using Workflow Modelling

#
=====
=====

# Run this once if DiagrammeR is not installed

install.packages("DiagrammeR")

# Load package

library(DiagrammeR)

# DevSecOps lifecycle workflow

grViz("
digraph DevSecOps {

graph [
  rankdir=LR,
  bgcolor=white,
  label='DevSecOps Lifecycle Workflow',
  labelloc=t,
  fontsize=20
]

```

```
node [  
    shape=box,  
    style='rounded, filled',  
    fontname=Helvetica,  
    fontsize=13  
]
```

```
Plan [  
    label='PLAN\\nRequirements & Security Planning',  
    fillcolor=lightblue  
]
```

```
Code [  
    label='CODE\\nSecure Coding',  
    fillcolor=lightgreen  
]
```

```
Build [  
    label='BUILD\\nCompile & Package',  
    fillcolor=lightyellow  
]
```

```
Test [  
    label='TEST\\nFunctional & Security Testing',  
    fillcolor=lightpink  
]
```

```
Security [  
    label='SECURITY\\nSAST / DAST / SCA',  
    fillcolor=lightcyan  
]
```

```
Deploy [  
    label='DEPLOY\\nSecure Deployment',  
    fillcolor=lavender  
]
```

```
Monitor [  
    label='MONITOR\\nSecurity Monitoring',  
    fillcolor=wheat  
]
```

```
Feedback [  
    label='FEEDBACK\\nRisk & Improvement',  
    shape=diamond,  
    fillcolor=lightgray  
]
```

Plan -> Code

Code -> Build

Build -> Test

Test -> Security

Security -> Deploy

Deploy -> Monitor

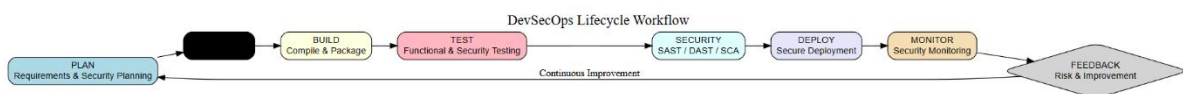
Monitor -> Feedback

Feedback -> Plan [label='Continuous Improvement']

}

")

Viewer Zoom



```

#
=====

=====

# EXPERIMENT 3

# Comparative Analysis of DevOps and DevSecOps using Synthetic Data

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


# Load packages

library(dygraphs)

library(xts)


# Synthetic performance data

days <- as.Date("2026-07-20") + 0:9


data <- xts(
  cbind(
    DevOps = c(62, 64, 66, 65, 68, 70, 71, 72, 74, 75),
    DevSecOps = c(60, 64, 69, 72, 75, 78, 81, 84, 87, 90)
  ),
  order.by = days
)

```

```
# Interactive comparison graph
```

```
dygraph(
```

```
  data,
```

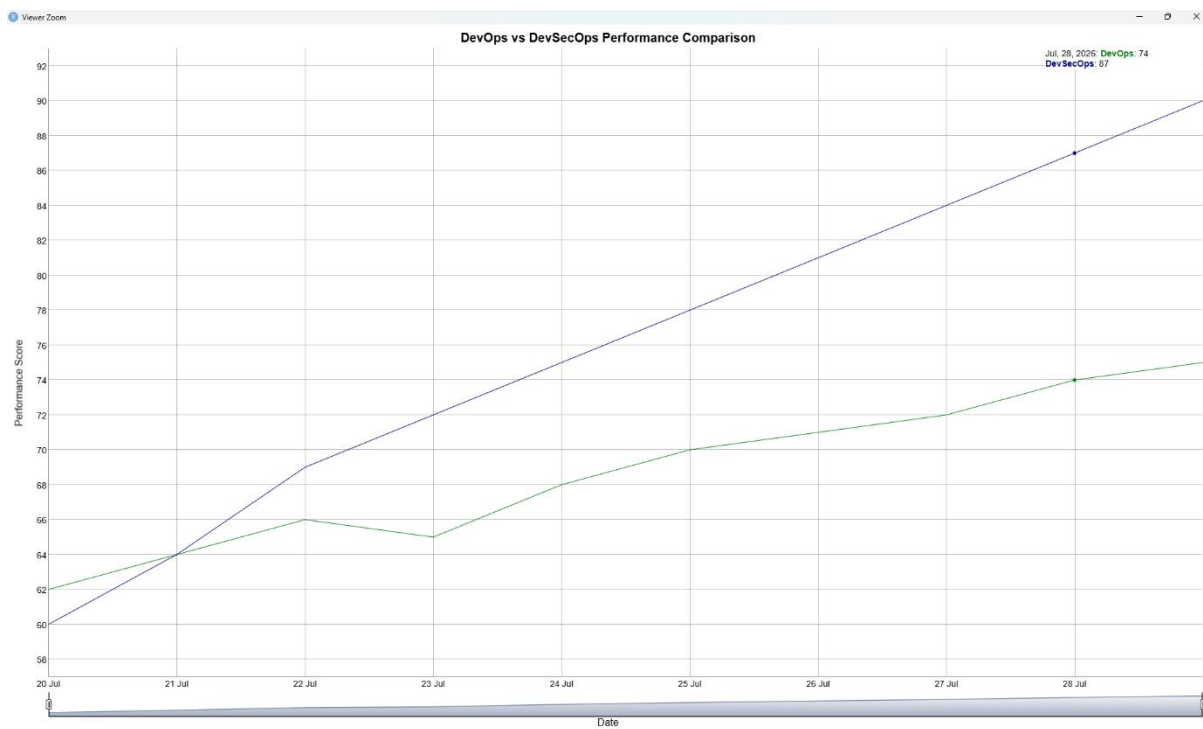
```
  main = "DevOps vs DevSecOps Performance Comparison",
```

```
  xlab = "Date",
```

```
  ylab = "Performance Score"
```

```
) %>%
```

```
  dyRangeSelector()
```



```

#
=====

=====

# EXPERIMENT 4

# Security Risk Assessment and Risk Matrix Simulation

#
=====

=====

# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic security risk data

days <- as.Date("2026-07-27") + 0:9


risk_data <- xts(
  cbind(
    ThreatLikelihood = c(3, 4, 5, 6, 7, 6, 5, 4, 3, 2),
    Impact = c(2, 3, 4, 5, 6, 7, 6, 5, 4, 3),
    RiskScore = c(6, 12, 20, 30, 42, 42, 30, 20, 12, 6)
  ),
  order.by = days
)

```

```
# Interactive security risk simulation
```

```
dygraph(
```

```
  risk_data,
```

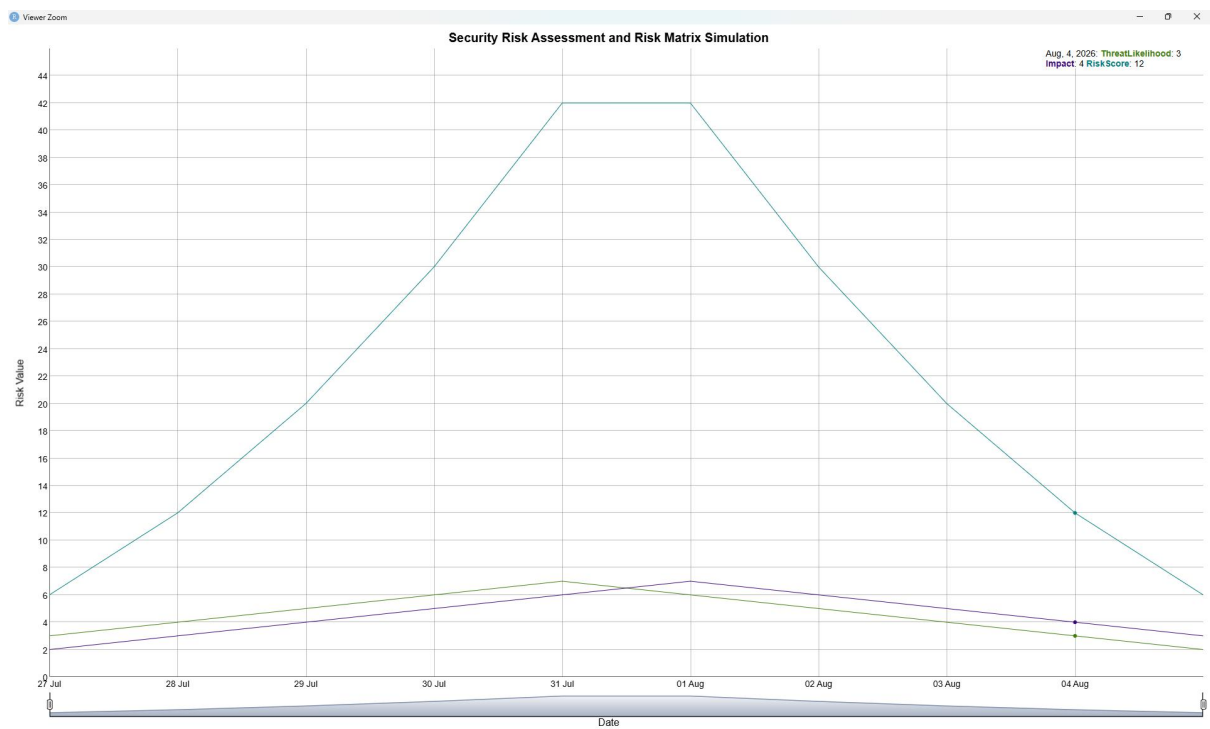
```
  main = "Security Risk Assessment and Risk Matrix Simulation",
```

```
  xlab = "Date",
```

```
  ylab = "Risk Value"
```

```
) %>%
```

```
  dyRangeSelector()
```



```

#
=====

=====

# EXPERIMENT 5

# Threat Propagation Simulation and Attack Surface Analysis

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic threat propagation data

days <- as.Date("2026-08-03") + 0:9


threat_data <- xts(
  cbind(
    ThreatLevel = c(2, 4, 7, 11, 16, 14, 11, 8, 5, 3),
    AttackSurface = c(10, 14, 19, 26, 34, 31, 27, 22, 17, 13),
    ExposedAssets = c(3, 5, 8, 12, 16, 14, 11, 8, 5, 4)
  ),
  order.by = days
)

```

```
# Interactive threat propagation analysis
```

```
dygraph(
```

```
  threat_data,
```

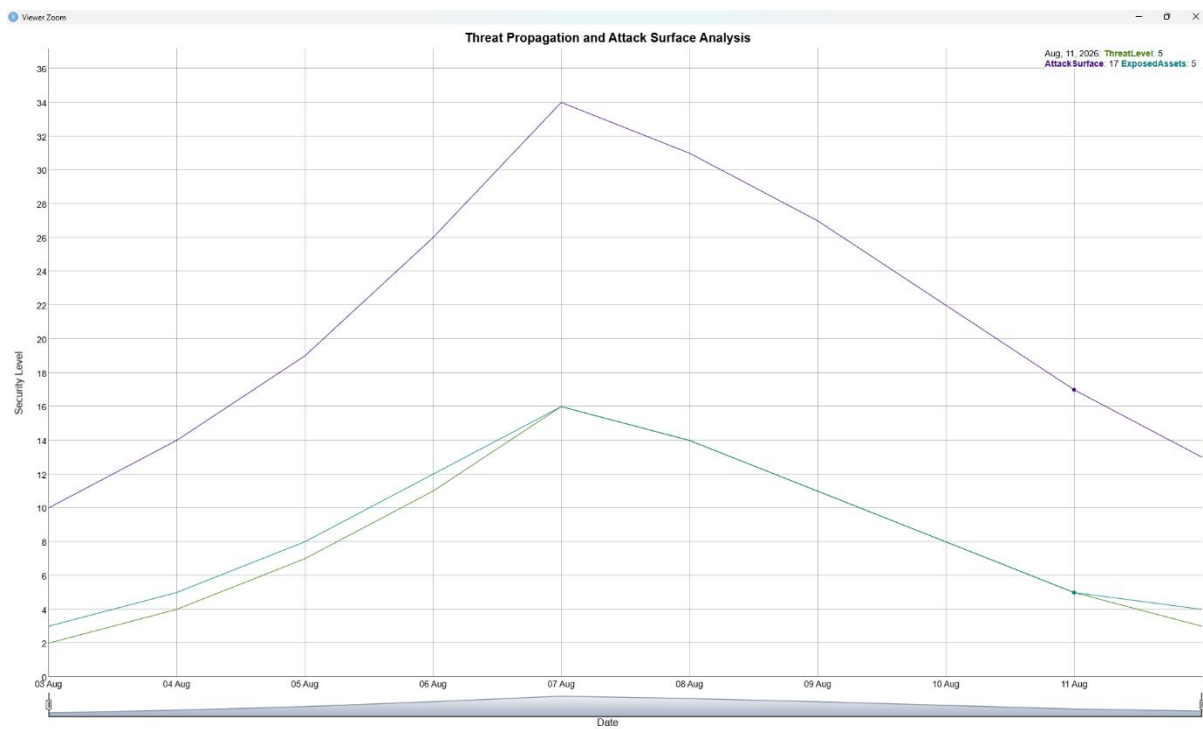
```
  main = "Threat Propagation and Attack Surface Analysis",
```

```
  xlab = "Date",
```

```
  ylab = "Security Level"
```

```
) %>%
```

```
  dyRangeSelector()
```



```

#
=====

=====

# EXPERIMENT 6

# Design and Simulation of Shift-Left Security Workflow

#
=====

=====

# Run once if required

install.packages("DiagrammeR")

library(DiagrammeR)

grViz("
digraph ShiftLeftSecurity {

graph [
  rankdir=LR,
  bgcolor=white,
  label='Shift-Left Security Workflow',
  labelloc=t,
  fontsize=20
]

node [
  shape=box,

```



```
style='rounded,filled',  
fontname=Helvetica,  
fontsize=13  
]
```

```
Plan [  
    label='Security Planning',  
    fillcolor=lightblue  
]
```

```
Code [  
    label='Developer Writes Code',  
    fillcolor=lightgreen  
]
```

```
SAST [  
    label='Early Security Check\\n(SAST)',  
    fillcolor=lightyellow  
]
```

```
Fix [  
    label='Fix Security Issues',  
    fillcolor=lightpink  
]
```

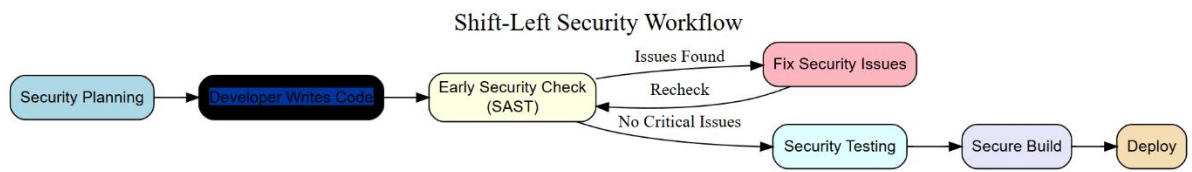
```
Test [  
    label='Test for Security Issues',  
    fillcolor=lightblue  
]
```

```
    label='Security Testing',  
    fillcolor=lightcyan  
]
```

```
Build [  
    label='Secure Build',  
    fillcolor=lavender  
]
```

```
Deploy [  
    label='Deploy',  
    fillcolor=wheat  
]
```

```
Plan -> Code  
Code -> SAST  
SAST -> Fix [label='Issues Found']  
Fix -> SAST [label='Recheck']  
SAST -> Test [label='No Critical Issues']  
Test -> Build  
Build -> Deploy  
}  
")
```



```

#
=====

=====

# EXPERIMENT 7

# Static Application Security Testing (SAST)
# and Security Verification

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic SAST scan results

days <- as.Date("2026-08-10") + 0:9


sast_data <- xts(
  cbind(
    Critical = c(4, 4, 3, 3, 2, 2, 1, 1, 0, 0),
    High = c(8, 7, 7, 6, 5, 4, 3, 2, 2, 1),
    Medium = c(12, 11, 10, 9, 8, 7, 6, 5, 4, 3),
    Low = c(15, 14, 13, 12, 11, 10, 9, 8, 7, 6)
  ),

```

```
order.by = days
```

```
)
```

```
# Interactive SAST vulnerability trend
```

```
dygraph(
```

```
  sast_data,
```

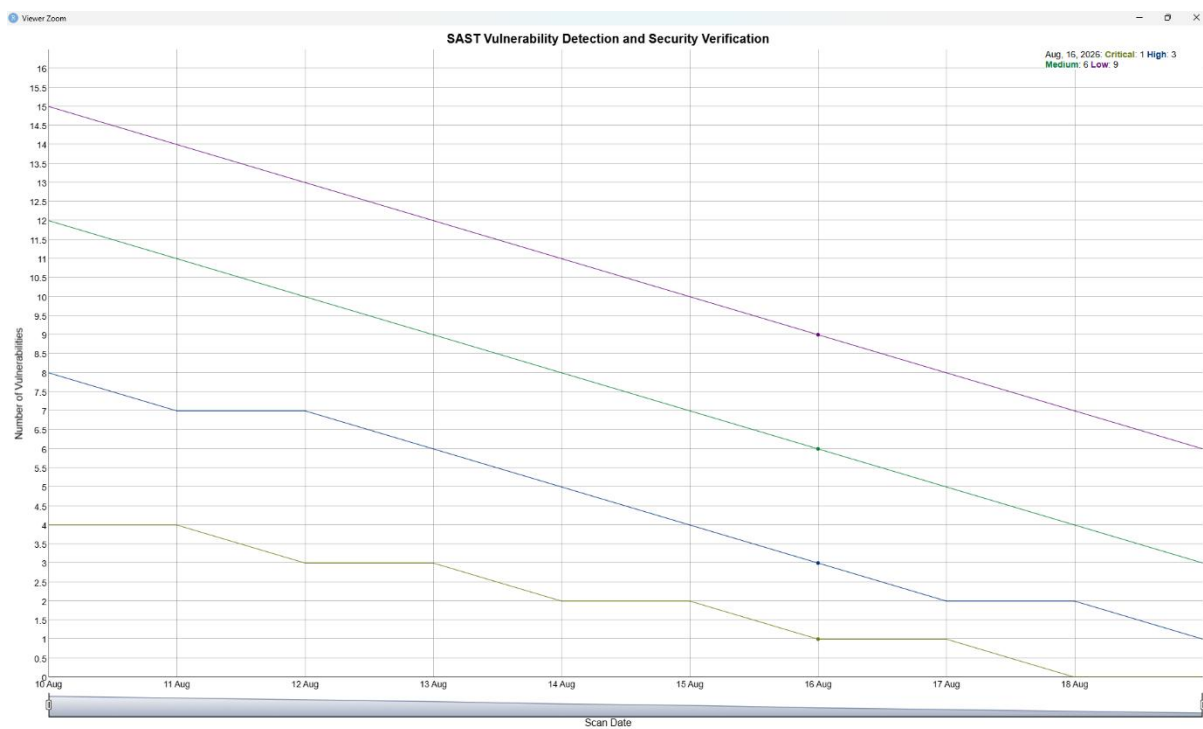
```
  main = "SAST Vulnerability Detection and Security Verification",
```

```
  xlab = "Scan Date",
```

```
  ylab = "Number of Vulnerabilities"
```

```
) %>%
```

```
  dyRangeSelector()
```



```

#
=====

=====

# EXPERIMENT 8

# Dynamic Application Security Testing (DAST)

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic DAST scan results

days <- as.Date("2026-08-17") + 0:9

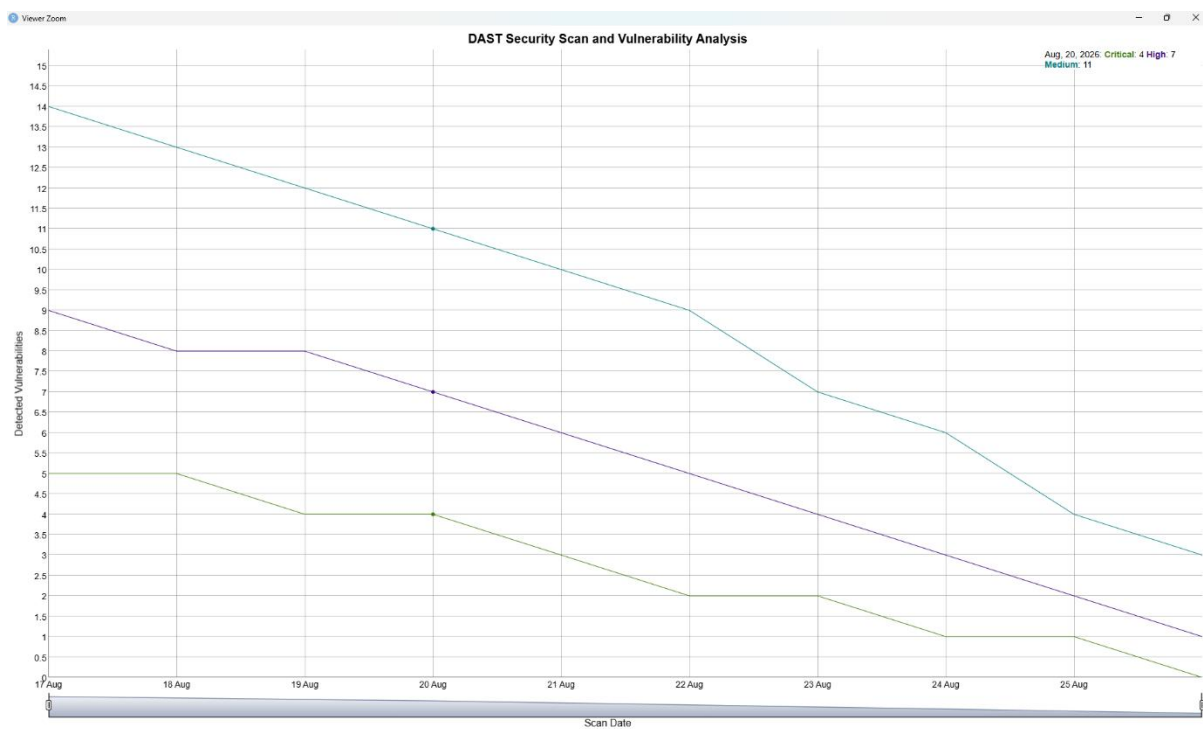

dast_data <- xts(
  cbind(
    Critical = c(5, 5, 4, 4, 3, 2, 2, 1, 1, 0),
    High = c(9, 8, 8, 7, 6, 5, 4, 3, 2, 1),
    Medium = c(14, 13, 12, 11, 10, 9, 7, 6, 4, 3)
  ),
  order.by = days
)

```

```

dygraph(
  dast_data,
  main = "DAST Security Scan and Vulnerability Analysis",
  xlab = "Scan Date",
  ylab = "Detected Vulnerabilities"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 9

# Software Composition Analysis (SCA)

#
=====

=====

# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic dependency vulnerability data

days <- as.Date("2026-08-24") + 0:9


sca_data <- xts(
  cbind(
    VulnerableDependencies = c(12, 11, 10, 9, 8, 7, 5, 4, 2, 1),
    OutdatedDependencies = c(8, 8, 7, 6, 6, 5, 4, 3, 2, 2),
    SecureDependencies = c(20, 21, 23, 25, 26, 28, 31, 33, 36, 38)
  ),
  order.by = days
)

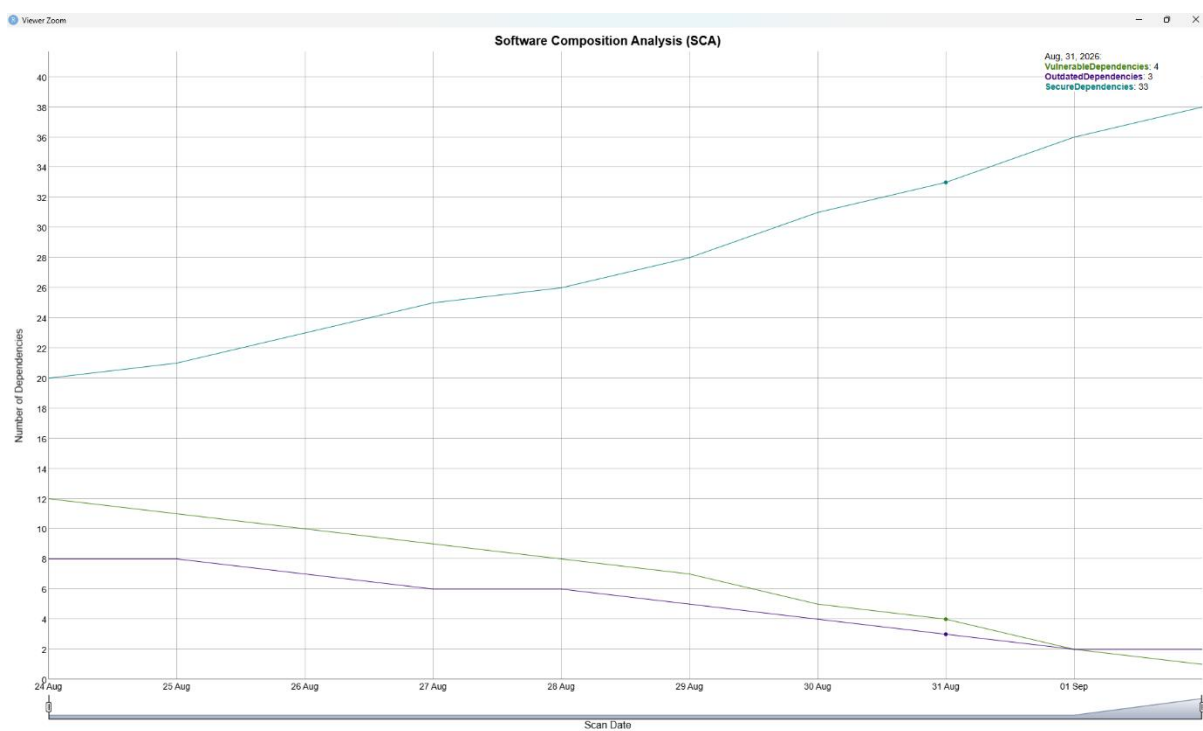
```



```

dygraph(
  sca_data,
  main = "Software Composition Analysis (SCA)",
  xlab = "Scan Date",
  ylab = "Number of Dependencies"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 10

# Vulnerability Trend Analysis and Security Reporting

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic vulnerability trend data

days <- as.Date("2026-08-31") + 0:9


vulnerability_data <- xts(
  cbind(
    Critical = c(8, 7, 6, 5, 4, 3, 3, 2, 1, 1),
    High = c(14, 13, 12, 11, 10, 8, 7, 6, 4, 3),
    Medium = c(20, 19, 18, 16, 15, 14, 12, 10, 8, 6)
  ),
  order.by = days
)

```

```
# Interactive vulnerability trend report
```

```
dygraph(
```

```
  vulnerability_data,
```

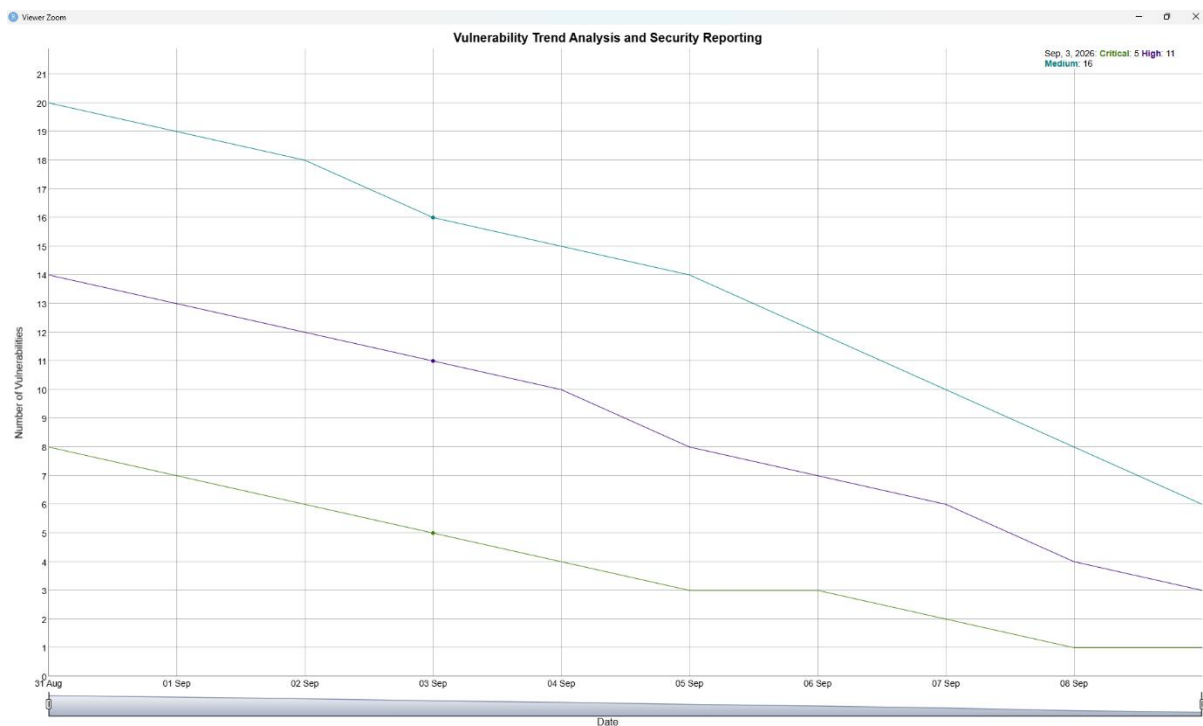
```
  main = "Vulnerability Trend Analysis and Security Reporting",
```

```
  xlab = "Date",
```

```
  ylab = "Number of Vulnerabilities"
```

```
) %>%
```

```
dyRangeSelector()
```



```

#
=====

=====

# EXPERIMENT 11

# Secure Infrastructure Architecture Design using IaC Concepts

#
=====

=====

# Run once if required

install.packages("DiagrammeR")

library(DiagrammeR)

grViz("
digraph IaC {

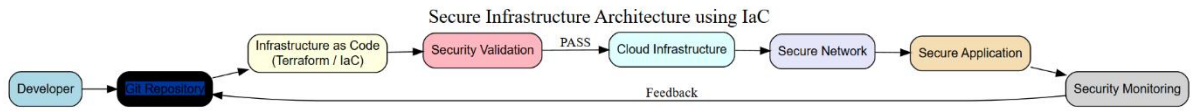
graph [
  rankdir=LR,
  bgcolor=white,
  label='Secure Infrastructure Architecture using IaC',
  labelloc=t,
  fontsize=20
]

node [
  shape=box,

```

```
    style='rounded,filled',
    fontname=Helvetica,
    fontsize=13
]
Developer [
    label='Developer',
    fillcolor=lightblue
]
Git [
    label='Git Repository',
    fillcolor=lightgreen
]
IaC [
    label='Infrastructure as Code\\n(Terraform / IaC)',
    fillcolor=lightyellow
]
Security [
    label='Security Validation',
    fillcolor=lightpink
]
Cloud [
    label='Cloud Infrastructure',
    fillcolor=lightcyan
]
Network [
    label='Secure Network',
```

```
    fillcolor=lavender
]
Application [
    label='Secure Application',
    fillcolor=wheat
]
Monitor [
    label='Security Monitoring',
    fillcolor=lightgray
]
Developer -> Git
Git -> IaC
IaC -> Security
Security -> Cloud [label='PASS']
Cloud -> Network
Network -> Application
Application -> Monitor
Monitor -> Git [label='Feedback']
}
")
```



```

#
=====

=====

# EXPERIMENT 12

# Identity and Access Management (IAM) Relationship Modelling

#
=====

=====

# Run once if required

install.packages("DiagrammeR")

library(DiagrammeR)

grViz("
digraph IAM {

graph [
  rankdir=LR,
  bgcolor=white,
  label='Identity and Access Management (IAM) Model',
  labelloc=t,
  fontsize=20
]

node [
  shape=box,

```



```
style='rounded,filled',  
fontname=Helvetica,  
fontsize=13  
]
```

```
User [  
    label='Users',  
    fillcolor=lightblue  
]
```

```
Admin [  
    label='Administrator',  
    fillcolor=lightgreen  
]
```

```
Developer [  
    label='Developer',  
    fillcolor=lightyellow  
]
```

```
Role [  
    label='Roles',  
    fillcolor=lightpink  
]
```

```
Permission [  
    label='Permissions',  
    fillcolor=lightblue  
]
```

```
    label='Permissions',  
    fillcolor=lightcyan  
]
```

```
Resource [  
    label='Resources',  
    fillcolor=lavender  
]
```

```
Audit [  
    label='Access Audit\\n& Monitoring',  
    fillcolor=wheat  
]
```

```
User -> Admin [label='assigned']
```

```
User -> Developer [label='assigned']
```

```
Admin -> Role [label='has role']
```

```
Developer -> Role [label='has role']
```

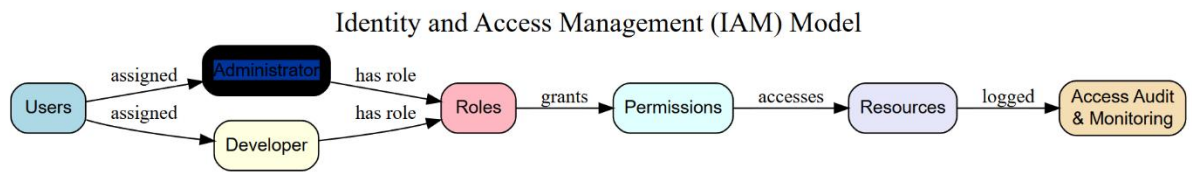
```
Role -> Permission [label='grants']
```

```
Permission -> Resource [label='accesses']
```

```
Resource -> Audit [label='logged']
```

```
}
```

```
")
```



```

#
=====

=====

# EXPERIMENT 13

# GitOps Workflow Design and Secure Deployment Simulation

#
=====

=====

# Run once if required

install.packages("DiagrammeR")

library(DiagrammeR)

grViz("
digraph GitOps {

graph [
  rankdir=LR,
  bgcolor=white,
  label='GitOps Secure Deployment Workflow',
  labelloc=t,
  fontsize=20
]

node [
  shape=box,

```

```
style='rounded,filled',  
fontname=Helvetica,  
fontsize=13  
]
```

```
Developer [  
    label='Developer\\nCode Changes',  
    fillcolor=lightblue  
]
```

```
Git [  
    label='Git Repository\\nVersion Control',  
    fillcolor=lightgreen  
]
```

```
CI [  
    label='CI Pipeline\\nBuild & Test',  
    fillcolor=lightyellow  
]
```

```
Security [  
    label='Security Checks\\nSAST / SCA',  
    fillcolor=lightpink  
]
```

```
CD [  
    label='CD Pipeline\\nDeploy',  
    fillcolor=lightblue  
]
```

```
    label='CD Pipeline\\nDeployment',  
    fillcolor=lightcyan  
]
```

```
Kubernetes [  
    label='Kubernetes\\nCluster',  
    fillcolor=lavender  
]
```

```
Monitor [  
    label='Monitoring\\n& Security Verification',  
    fillcolor=wheat  
]
```

Developer -> Git

Git -> CI

CI -> Security

Security -> CD [label='PASS']

Security -> Git [label='FAIL']

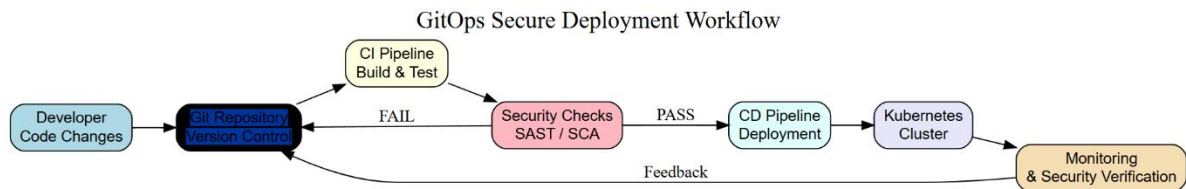
CD -> Kubernetes

Kubernetes -> Monitor

Monitor -> Git [label='Feedback']

```
}
```

```
")
```



#

=====

=====

EXPERIMENT 14

Container Security and Vulnerability Assessment

#

=====

=====

Run once if required

install.packages("dygraphs")

install.packages("xts")

library(dygraphs)

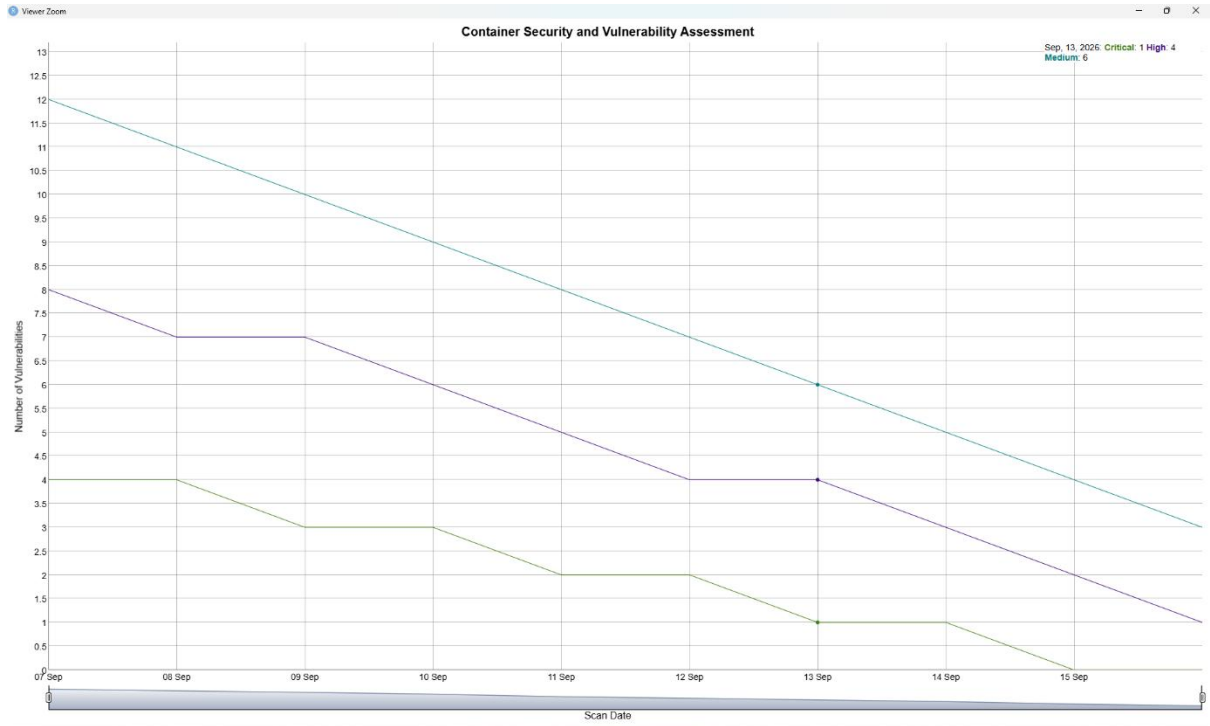
library(xts)

Synthetic container security scan data

```
days <- as.Date("2026-09-07") + 0:9
```

```
container_data <- xts(  
  cbind(  
    Critical = c(4, 4, 3, 3, 2, 2, 1, 1, 0, 0),  
    High = c(8, 7, 7, 6, 5, 4, 4, 3, 2, 1),  
    Medium = c(12, 11, 10, 9, 8, 7, 6, 5, 4, 3)  
  ),  
  order.by = days  
)
```

```
dygraph(  
  container_data,  
  main = "Container Security and Vulnerability Assessment",  
  xlab = "Scan Date",  
  ylab = "Number of Vulnerabilities"  
) %>%  
  dyRangeSelector()
```

```

#
=====

=====

# EXPERIMENT 15

# Security Log Generation, Incident Timeline Analysis
# and Anomaly Detection

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic security log data

time <- as.POSIXct("2026-09-12 09:00:00") + (0:9) * 3600


log_data <- xts(
  cbind(
    NormalEvents = c(8, 10, 9, 11, 10, 9, 12, 10, 9, 11),
    FailedLogins = c(2, 3, 2, 4, 3, 2, 3, 2, 3, 2),
    SuspiciousEvents = c(0, 1, 0, 1, 2, 12, 1, 0, 10, 1)
  ),
  order.by = time

```

)

Interactive security log and anomaly analysis

dygraph(

log_data,

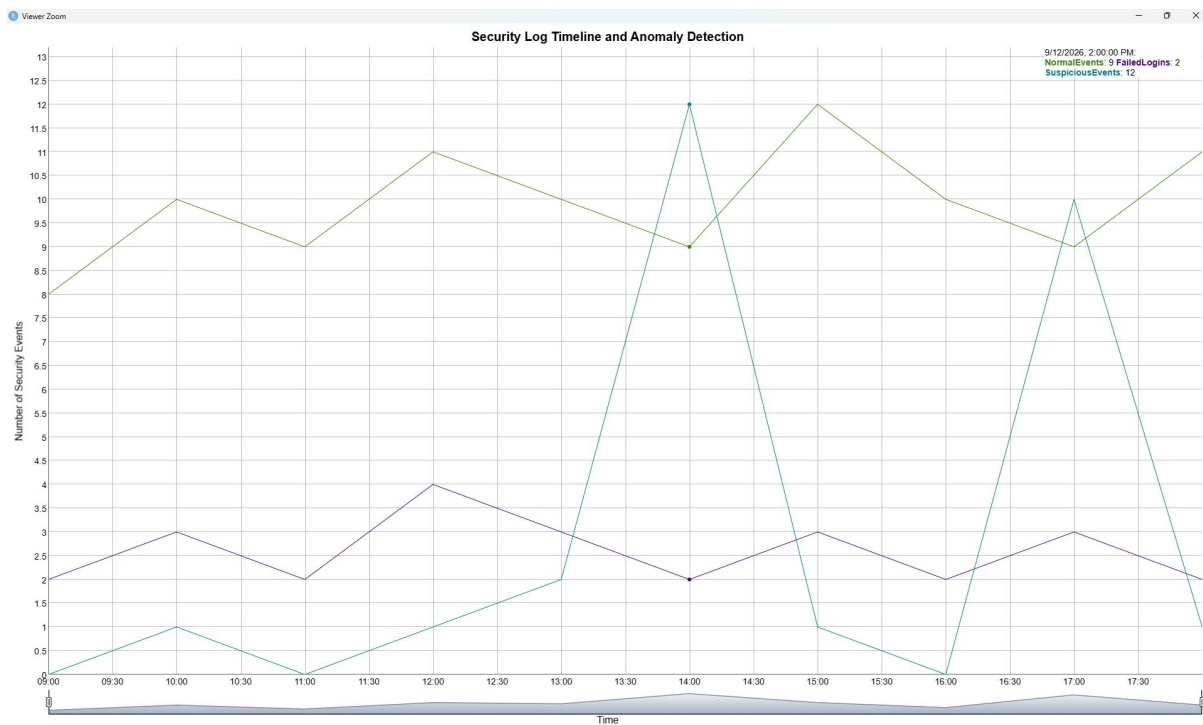
main = "Security Log Timeline and Anomaly Detection",

xlab = "Time",

ylab = "Number of Security Events"

) %>%

dyRangeSelector()



```

#
=====

=====

# EXPERIMENT 16

# Development of a DevSecOps Security Monitoring Dashboard

#
=====

=====

# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic security monitoring data

days <- as.Date("2026-09-17") + 0:9

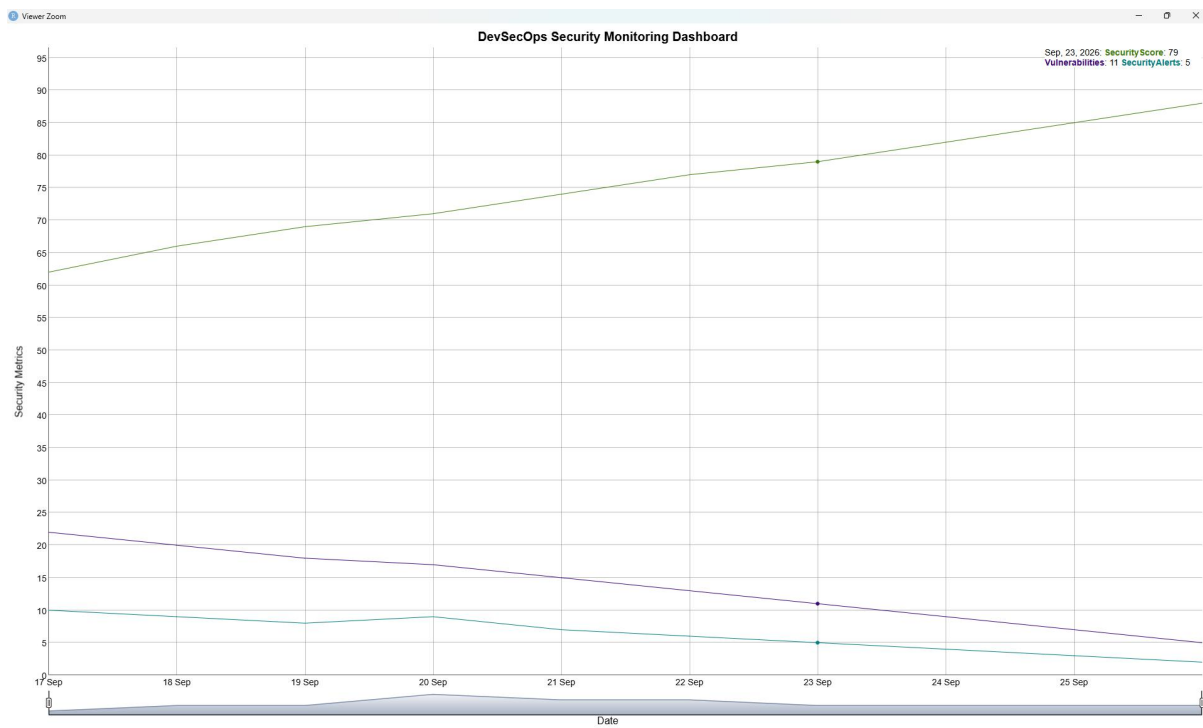

dashboard_data <- xts(
  cbind(
    SecurityScore = c(62, 66, 69, 71, 74, 77, 79, 82, 85, 88),
    Vulnerabilities = c(22, 20, 18, 17, 15, 13, 11, 9, 7, 5),
    SecurityAlerts = c(10, 9, 8, 9, 7, 6, 5, 4, 3, 2)
  ),
  order.by = days
)

```

```

dygraph(
  dashboard_data,
  main = "DevSecOps Security Monitoring Dashboard",
  xlab = "Date",
  ylab = "Security Metrics"
) %>%
  dyRangeSelector()

```



```

#
=====
=====

# EXPERIMENT 17

# Secure Coding Practices and Vulnerability Detection Simulation

#
=====
=====


# Run once if required

install.packages("dygraphs")
install.packages("xts")


library(dygraphs)
library(xts)


# Synthetic secure coding scan data
days <- as.Date("2026-09-19") + 0:9

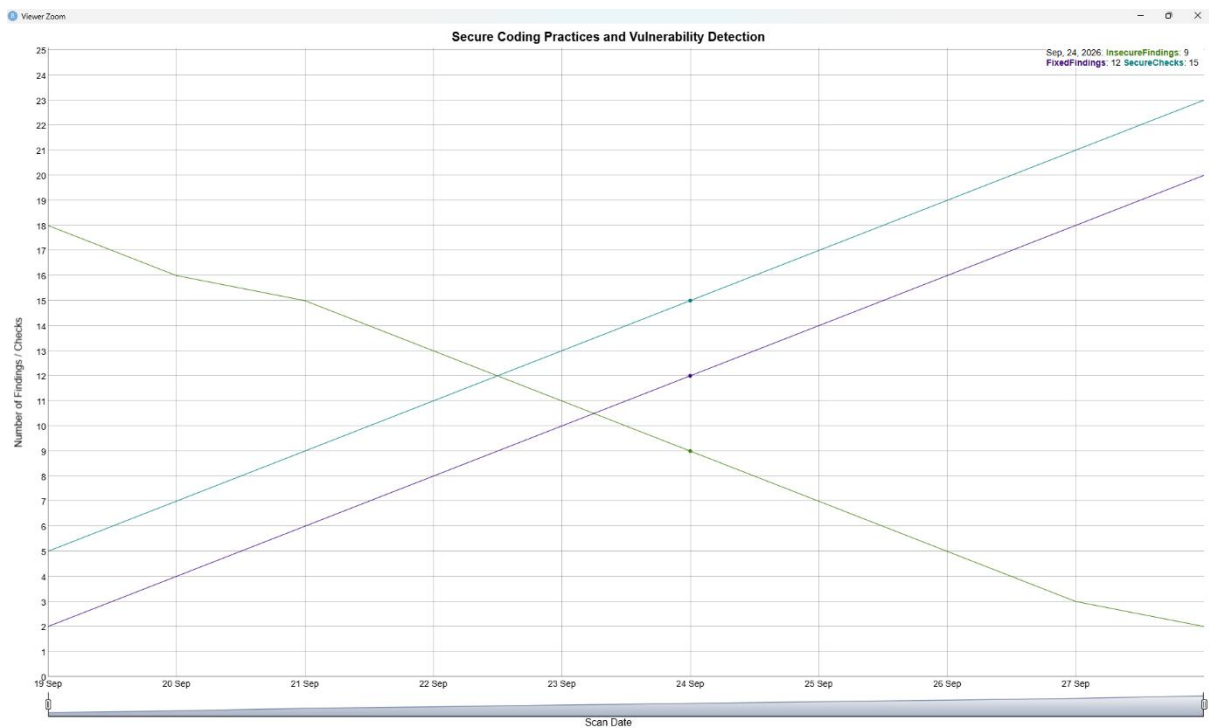

coding_data <- xts(
  cbind(
    InsecureFindings = c(18, 16, 15, 13, 11, 9, 7, 5, 3, 2),
    FixedFindings = c(2, 4, 6, 8, 10, 12, 14, 16, 18, 20),
    SecureChecks = c(5, 7, 9, 11, 13, 15, 17, 19, 21, 23)
  ),
  order.by = days
)

```

```

dygraph(
  coding_data,
  main = "Secure Coding Practices and Vulnerability Detection",
  xlab = "Scan Date",
  ylab = "Number of Findings / Checks"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 18

# OWASP Top 10 Vulnerability Identification and Risk Classification

#
=====

=====

# Run once if required

install.packages("DiagrammeR")

library(DiagrammeR)

grViz("
digraph OWASP {

graph [
  rankdir=LR,
  bgcolor=white,
  label='OWASP Top 10 Vulnerability Identification and Risk Classification',
  labelloc=t,
  fontsize=20
]

node [
  shape=box,

```



```
style='rounded,filled',  
fontname=Helvetica,  
fontsize=12  
]
```

```
App [  
    label='Web Application',  
    fillcolor=lightblue  
]
```

```
A01 [  
    label='A01 Broken Access Control',  
    fillcolor=lightpink  
]
```

```
A02 [  
    label='A02 Cryptographic Failures',  
    fillcolor=lightyellow  
]
```

```
A03 [  
    label='A03 Injection',  
    fillcolor=lightcyan  
]
```

```
A04 [  
    label='A04 Security Misconfiguration',  
    fillcolor=lightpink  
]
```

```
label='A04 Insecure Design',  
fillcolor=lavender  
]
```

```
A05 [  
label='A05 Security Misconfiguration',  
fillcolor=wheat  
]
```

```
A06 [  
label='A06 Vulnerable Components',  
fillcolor=lightgreen  
]
```

```
Risk [  
label='Risk Classification\\nCritical / High / Medium / Low',  
shape=diamond,  
fillcolor=lightgray  
]
```

App -> A01

App -> A02

App -> A03

App -> A04

App -> A05

App -> A06

A01 -> Risk

A02 -> Risk

A03 -> Risk

A04 -> Risk

A05 -> Risk

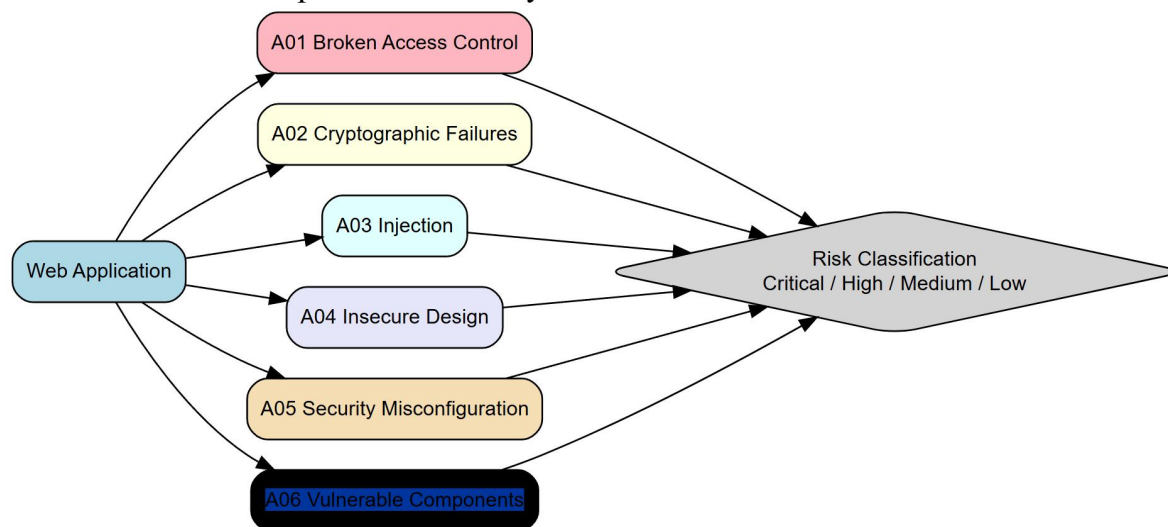
A06 -> Risk

}

")

Viewer Zoom

OWASP Top 10 Vulnerability Identification and Risk Classification



```

#
=====
=====

# EXPERIMENT 19

# Security Test Case Design and Automated Verification Simulation

#
=====
=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic automated security test results

days <- as.Date("2026-09-26") + 0:9

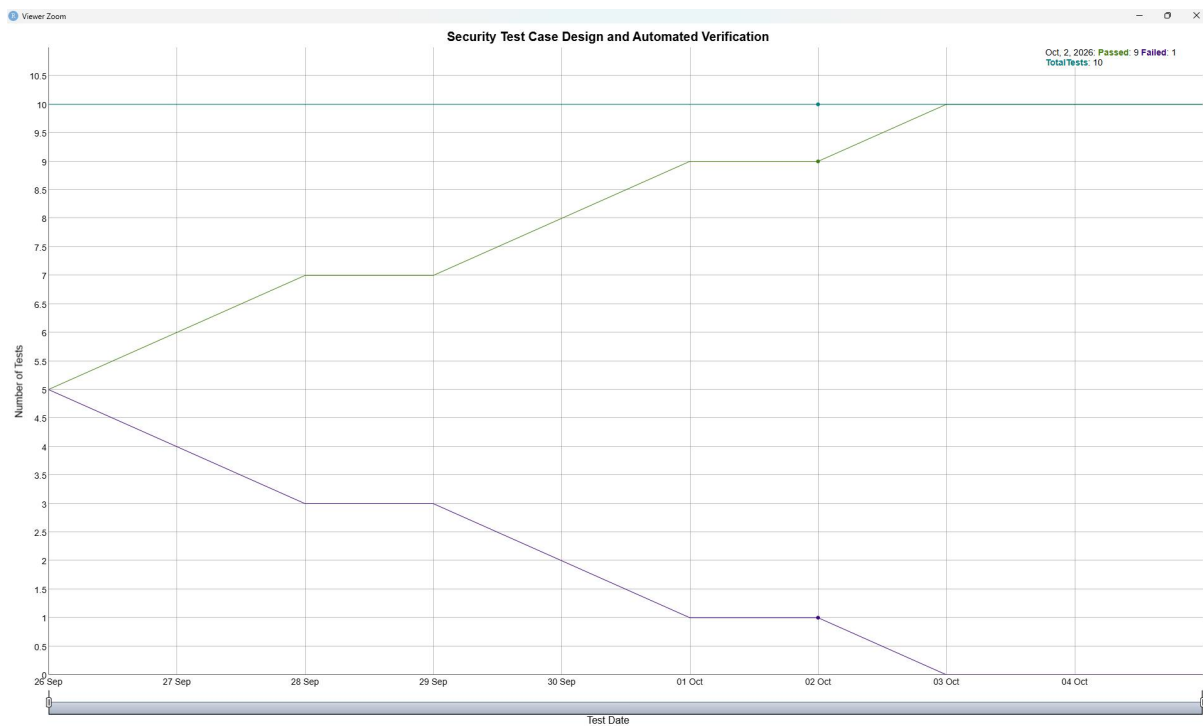

test_data <- xts(
  cbind(
    Passed = c(5, 6, 7, 7, 8, 9, 9, 10, 10, 10),
    Failed = c(5, 4, 3, 3, 2, 1, 1, 0, 0, 0),
    TotalTests = c(10, 10, 10, 10, 10, 10, 10, 10, 10, 10)
  ),
  order.by = days
)

```

```

dygraph(
  test_data,
  main = "Security Test Case Design and Automated Verification",
  xlab = "Test Date",
  ylab = "Number of Tests"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 20

# CI/CD Pipeline Security Verification and Quality Gate Simulation

#
=====

=====

# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic CI/CD security verification data

days <- as.Date("2026-09-28") + 0:9

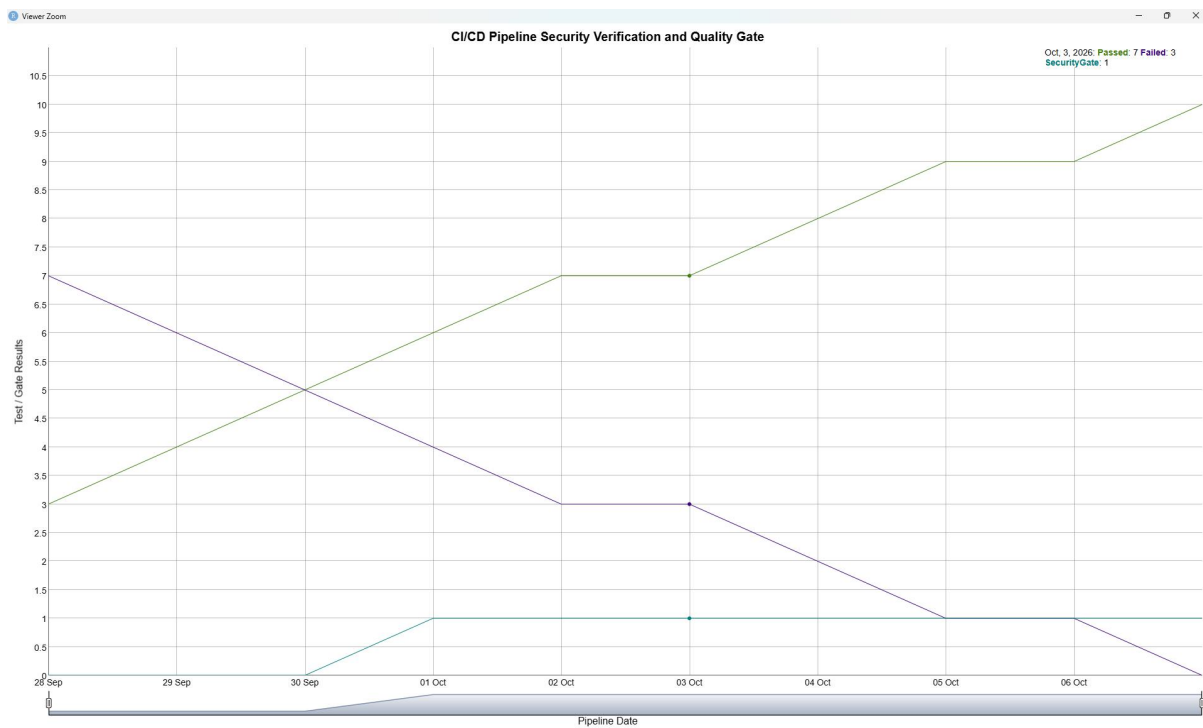

pipeline_data <- xts(
  cbind(
    Passed = c(3, 4, 5, 6, 7, 7, 8, 9, 9, 10),
    Failed = c(7, 6, 5, 4, 3, 3, 2, 1, 1, 0),
    SecurityGate = c(0, 0, 0, 1, 1, 1, 1, 1, 1, 1)
  ),
  order.by = days
)

```

```

dygraph(
  pipeline_data,
  main = "CI/CD Pipeline Security Verification and Quality Gate",
  xlab = "Pipeline Date",
  ylab = "Test / Gate Results"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 21

# Security Vulnerability Prioritization using CVSS-Based Scoring

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic CVSS vulnerability scores

days <- as.Date("2026-10-01") + 0:9


cvss_data <- xts(
  cbind(
    CVSS = c(3.2, 5.1, 6.4, 7.2, 8.1, 9.0, 6.8, 4.5, 8.7, 7.5),
    Priority = c(1, 2, 2, 3, 3, 4, 3, 2, 4, 3)
  ),
  order.by = days
)

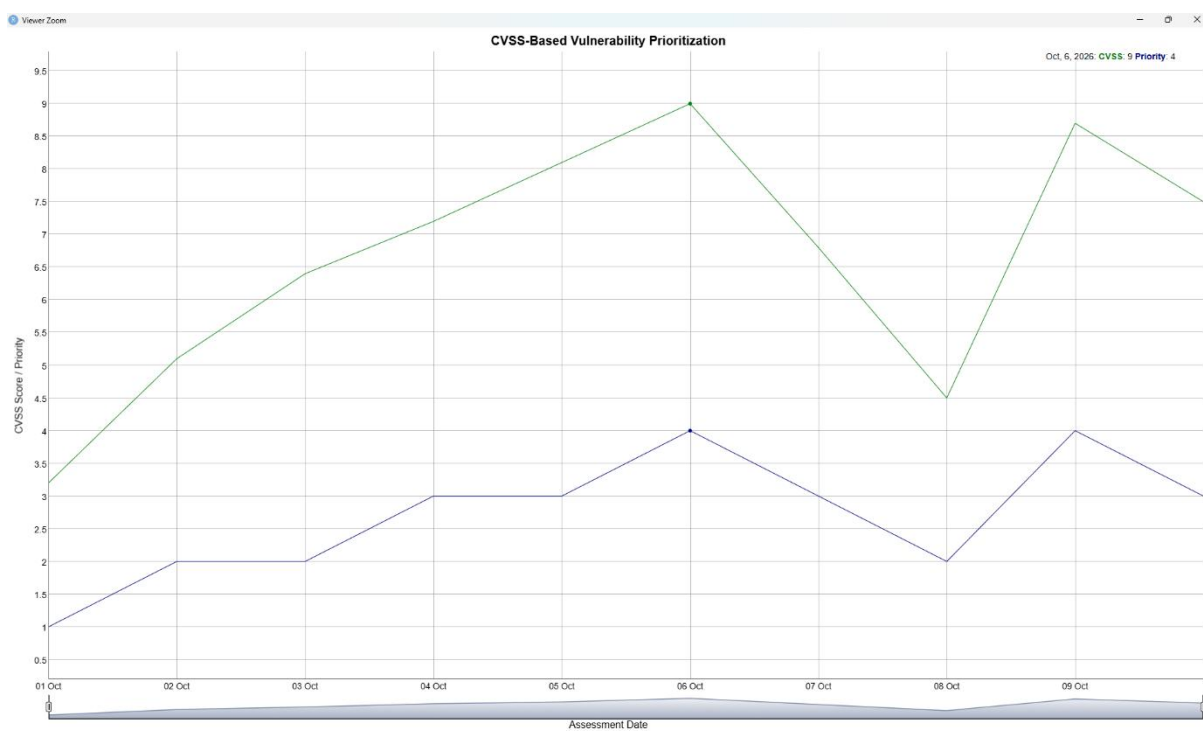
```



```

dygraph(
  cvss_data,
  main = "CVSS-Based Vulnerability Prioritization",
  xlab = "Assessment Date",
  ylab = "CVSS Score / Priority"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 22

# Secrets Detection and Secure Secrets Management Simulation

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic secrets detection data

days <- as.Date("2026-10-05") + 0:9

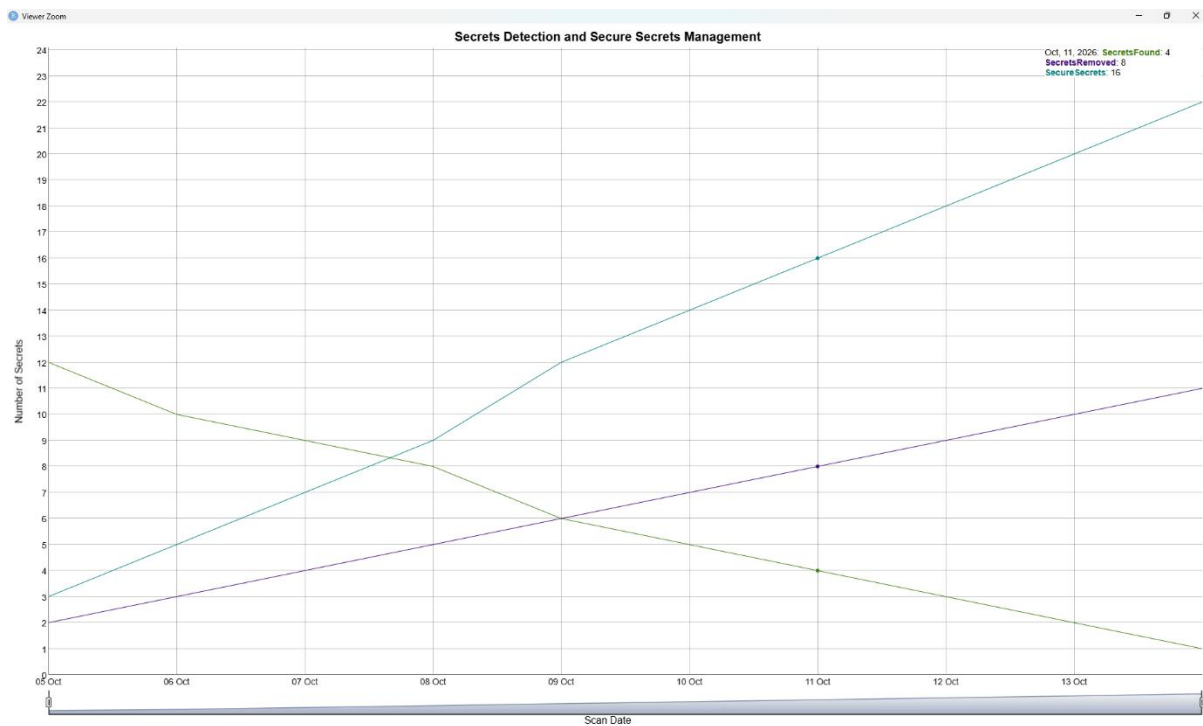

secrets_data <- xts(
  cbind(
    SecretsFound = c(12, 10, 9, 8, 6, 5, 4, 3, 2, 1),
    SecretsRemoved = c(2, 3, 4, 5, 6, 7, 8, 9, 10, 11),
    SecureSecrets = c(3, 5, 7, 9, 12, 14, 16, 18, 20, 22)
  ),
  order.by = days
)

```

```

dygraph(
  secrets_data,
  main = "Secrets Detection and Secure Secrets Management",
  xlab = "Scan Date",
  ylab = "Number of Secrets"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 23

# API Security Testing and Vulnerability Analysis

#
=====

=====

# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic API security testing data

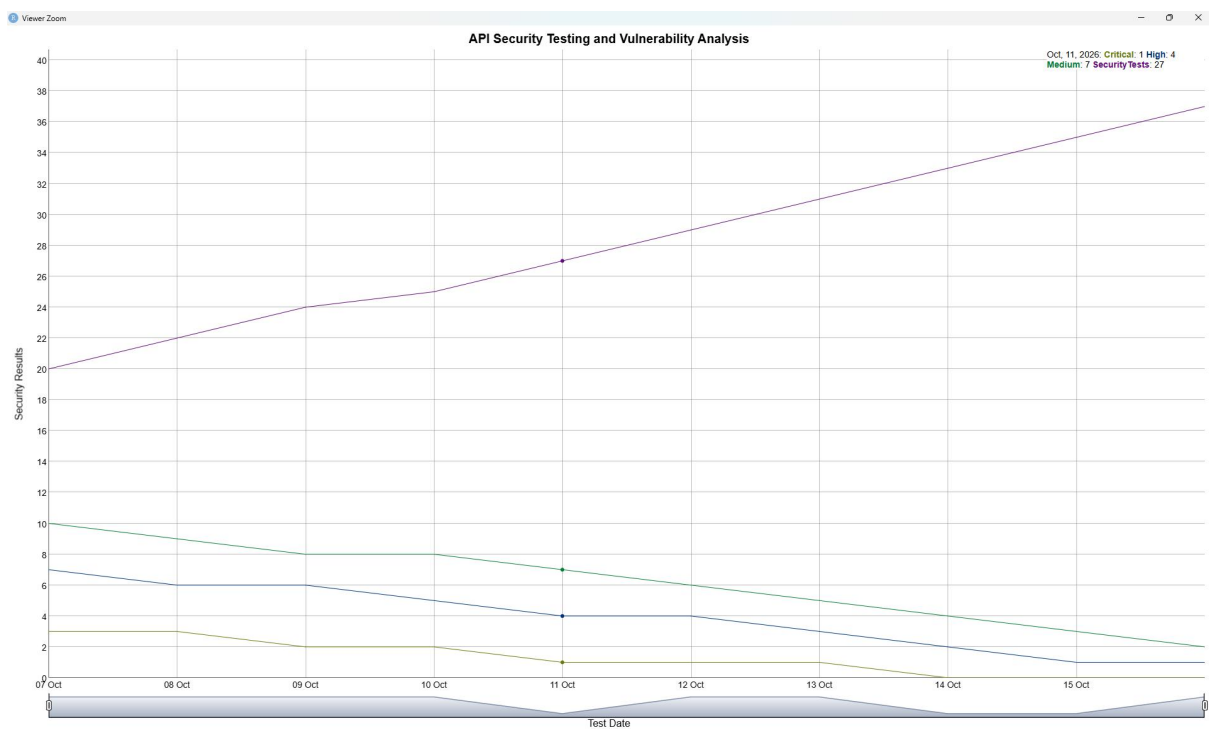
days <- as.Date("2026-10-07") + 0:9


api_data <- xts(
  cbind(
    Critical = c(3, 3, 2, 2, 1, 1, 1, 0, 0, 0),
    High = c(7, 6, 6, 5, 4, 4, 3, 2, 1, 1),
    Medium = c(10, 9, 8, 8, 7, 6, 5, 4, 3, 2),
    SecurityTests = c(20, 22, 24, 25, 27, 29, 31, 33, 35, 37)
  ),
  order.by = days

```

)

```
dygraph(  
  api_data,  
  main = "API Security Testing and Vulnerability Analysis",  
  xlab = "Test Date",  
  ylab = "Security Results"  
) %>%  
  dyRangeSelector()
```



```

#
=====
=====

# EXPERIMENT 24

# Web Application Security Vulnerability Simulation

#
=====
=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic web application vulnerability data

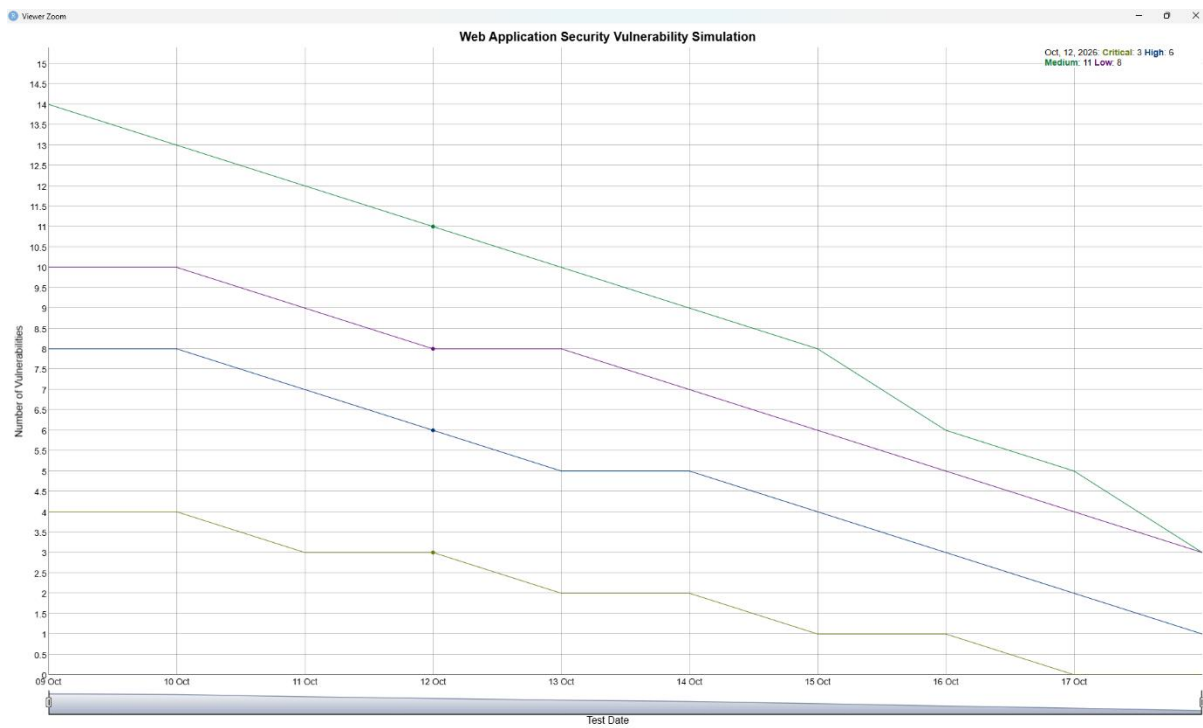
days <- as.Date("2026-10-09") + 0:9


web_data <- xts(
  cbind(
    Critical = c(4, 4, 3, 3, 2, 2, 1, 1, 0, 0),
    High = c(8, 8, 7, 6, 5, 5, 4, 3, 2, 1),
    Medium = c(14, 13, 12, 11, 10, 9, 8, 6, 5, 3),
    Low = c(10, 10, 9, 8, 8, 7, 6, 5, 4, 3)
  ),
  order.by = days
)

```

)

```
dygraph(  
  web_data,  
  main = "Web Application Security Vulnerability Simulation",  
  xlab = "Test Date",  
  ylab = "Number of Vulnerabilities"  
) %>%  
  dyRangeSelector()
```



```

#
=====

=====

# EXPERIMENT 25

# Security Policy Enforcement using Rule-Based Simulation

#
=====

=====

# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic security policy enforcement data

days <- as.Date("2026-10-12") + 0:9


policy_data <- xts(
  cbind(
    Allowed = c(8, 9, 10, 11, 12, 13, 14, 15, 16, 17),
    Blocked = c(7, 6, 6, 5, 4, 4, 3, 2, 2, 1),
    PolicyViolations = c(6, 5, 5, 4, 3, 3, 2, 2, 1, 1)
  ),
  order.by = days
)

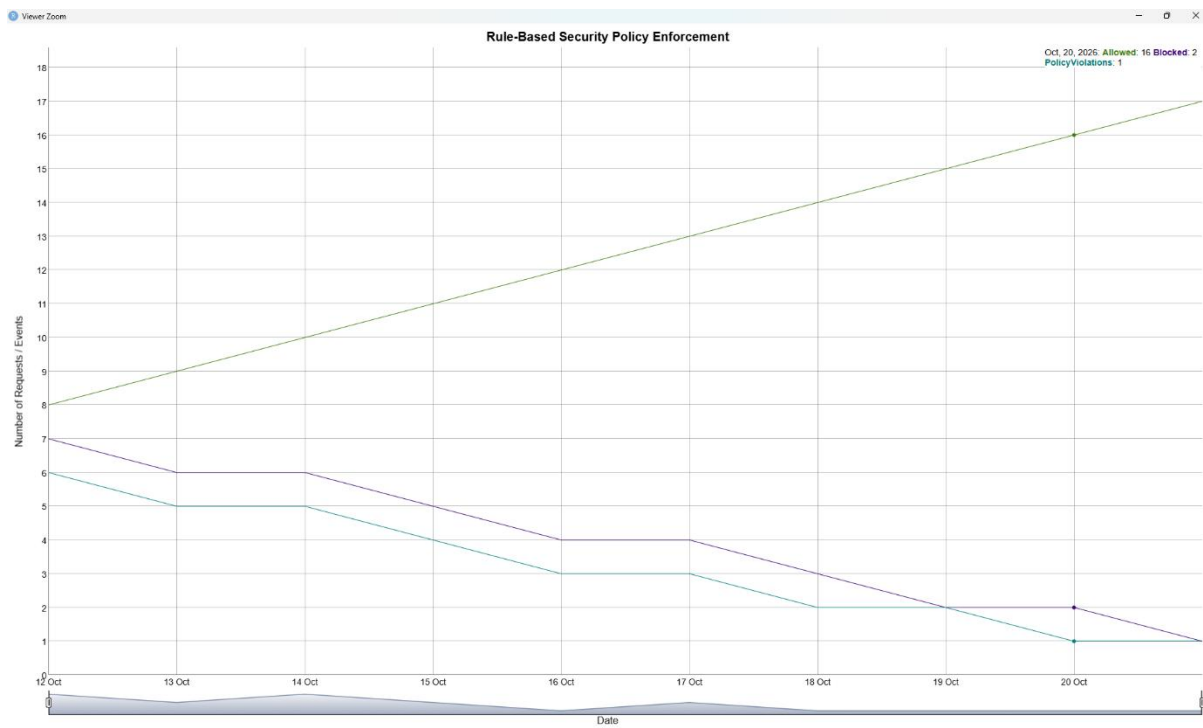
```



```

dygraph(
  policy_data,
  main = "Rule-Based Security Policy Enforcement",
  xlab = "Date",
  ylab = "Number of Requests / Events"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 26

# DevSecOps Security Gate Modelling and Decision Simulation

#
=====

=====

# Run once if required

install.packages("DiagrammeR")

library(DiagrammeR)

grViz("
digraph SecurityGate {

graph [
  rankdir=LR,
  bgcolor=white,
  label='DevSecOps Security Gate Decision Workflow',
  labelloc=t,
  fontsize=20
]

node [
  shape=box,

```

```
style='rounded,filled',  
fontname=Helvetica,  
fontsize=13  
]
```

```
Start [  
    label='Start',  
    fillcolor=lightblue  
]
```

```
Build [  
    label='Application Build',  
    fillcolor=lightgreen  
]
```

```
SecurityScan [  
    label='Security Scan',  
    fillcolor=lightyellow  
]
```

```
Gate [  
    label='Security Gate\\nDecision',  
    shape=diamond,  
    fillcolor=lightpink  
]
```

```
Deploy [  
    label='Deploy Application',  
    fillcolor=lightcyan  
]
```

```
Reject [  
    label='Reject Build\\n& Fix Issues',  
    fillcolor=wheat  
]
```

Start -> Build

Build -> SecurityScan

SecurityScan -> Gate

Gate -> Deploy [label='PASS']

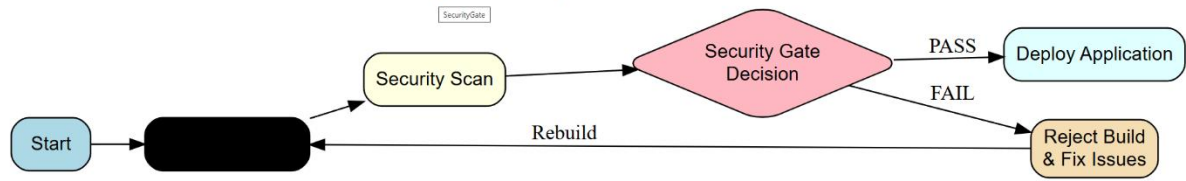
Gate -> Reject [label='FAIL']

Reject -> Build [label='Rebuild']

}

")

DevSecOps Security Gate Decision Workflow



```

#
=====
=====

# EXPERIMENT 27

# Security Metrics Generation and KPI Analysis

#
=====
=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic security KPI data

days <- as.Date("2026-10-14") + 0:9

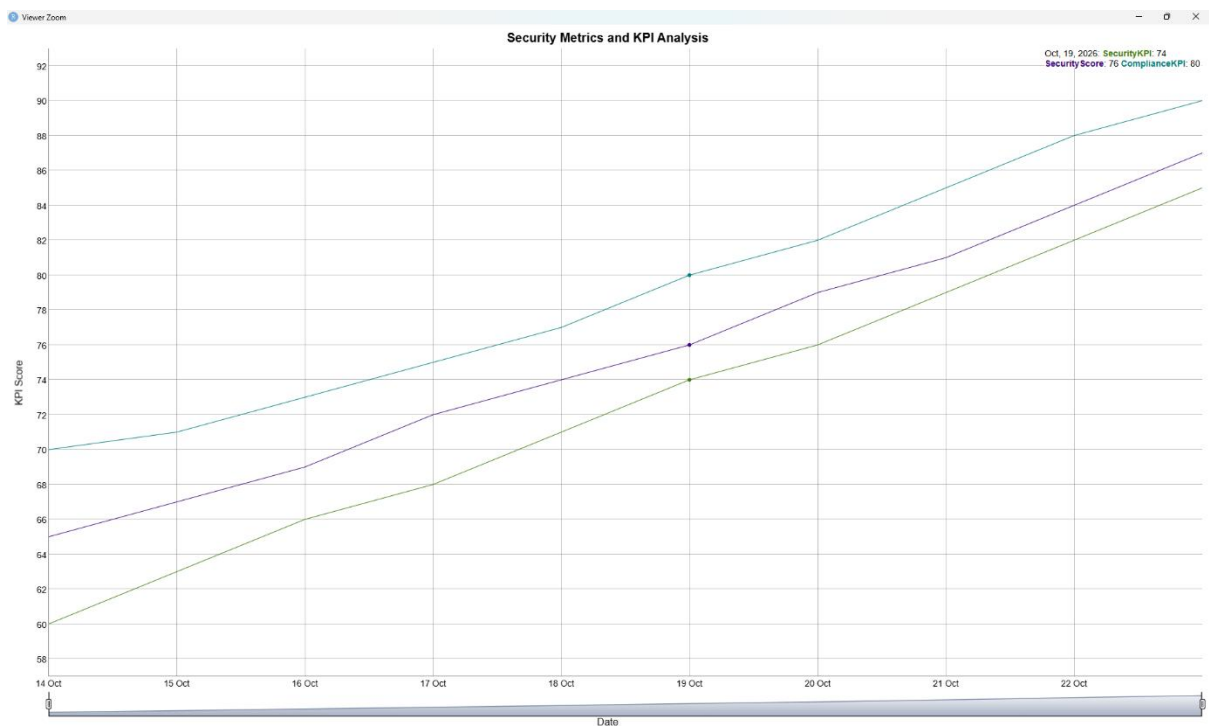

kpi_data <- xts(
  cbind(
    SecurityKPI = c(60, 63, 66, 68, 71, 74, 76, 79, 82, 85),
    SecurityScore = c(65, 67, 69, 72, 74, 76, 79, 81, 84, 87),
    ComplianceKPI = c(70, 71, 73, 75, 77, 80, 82, 85, 88, 90)
  ),
  order.by = days
)

```

```

dygraph(
  kpi_data,
  main = "Security Metrics and KPI Analysis",
  xlab = "Date",
  ylab = "KPI Score"
) %>%
  dyRangeSelector()

```



```

#
=====
=====

# EXPERIMENT 28

# Continuous Security Monitoring and Vulnerability Detection Simulation

#
=====
=====


# Run once if required

install.packages("dygraphs")
install.packages("xts")


library(dygraphs)
library(xts)


# Synthetic continuous security monitoring data
days <- as.Date("2026-10-19") + 0:9


monitoring_data <- xts(
  cbind(
    DetectedVulnerabilities = c(4, 6, 5, 8, 10, 7, 6, 5, 3, 2),
    SecurityAlerts = c(3, 5, 4, 7, 9, 6, 5, 4, 3, 2),
    ResolvedVulnerabilities = c(1, 2, 2, 3, 4, 4, 5, 5, 4, 3)
  ),
  order.by = days
)

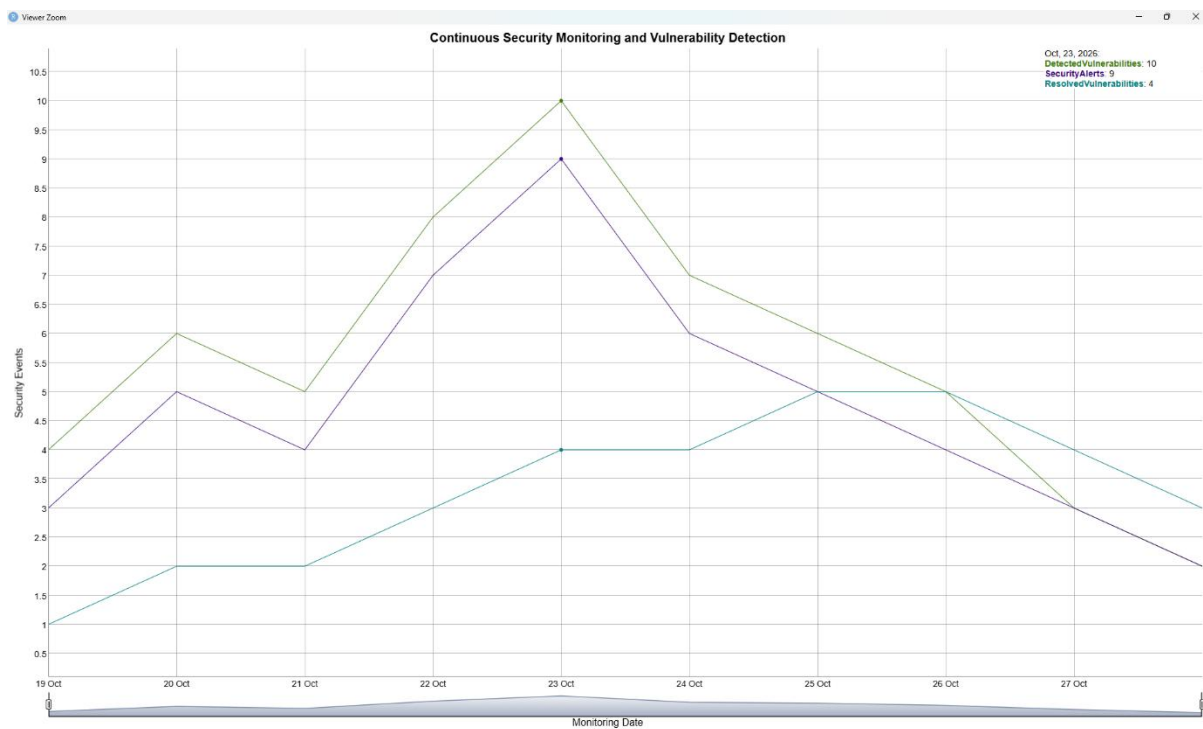
```



```

dygraph(
  monitoring_data,
  main = "Continuous Security Monitoring and Vulnerability Detection",
  xlab = "Monitoring Date",
  ylab = "Security Events"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 29

# Security Event Correlation and Attack Pattern Analysis

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic security event correlation data

days <- as.Date("2026-10-21") + 0:9

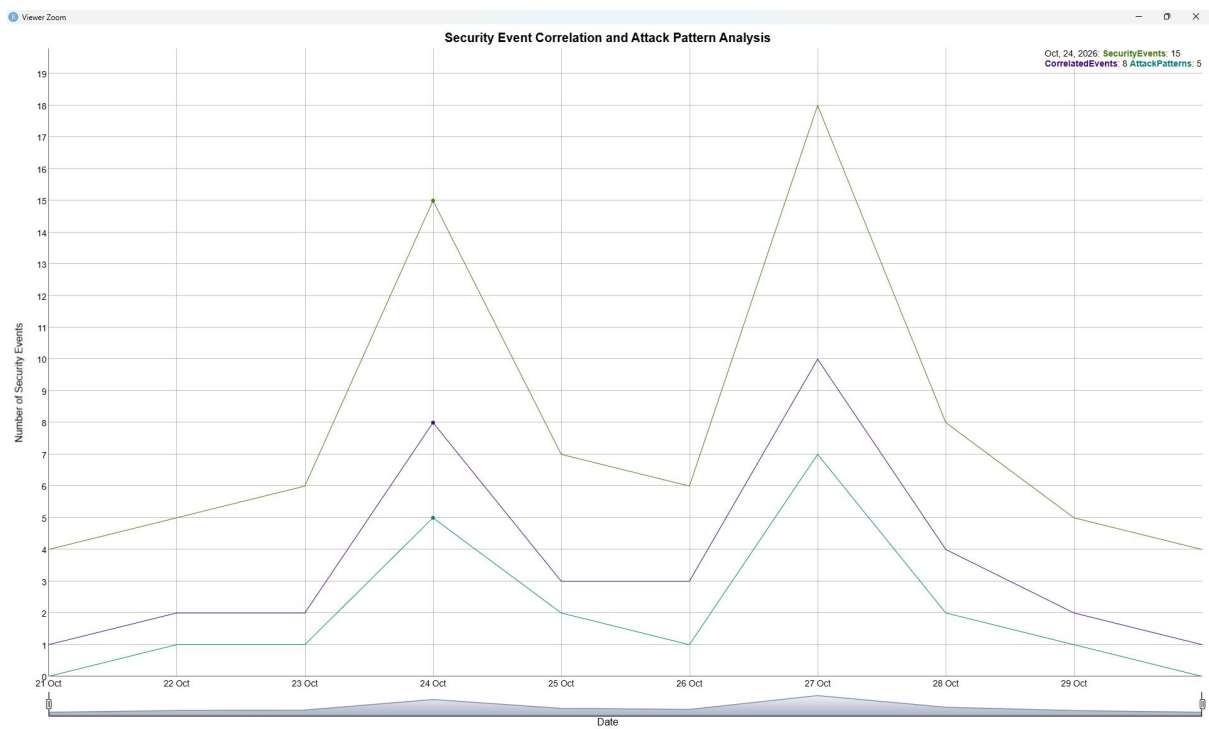

event_data <- xts(
  cbind(
    SecurityEvents = c(4, 5, 6, 15, 7, 6, 18, 8, 5, 4),
    CorrelatedEvents = c(1, 2, 2, 8, 3, 3, 10, 4, 2, 1),
    AttackPatterns = c(0, 1, 1, 5, 2, 1, 7, 2, 1, 0)
  ),
  order.by = days
)

```

```

dygraph(
  event_data,
  main = "Security Event Correlation and Attack Pattern Analysis",
  xlab = "Date",
  ylab = "Number of Security Events"
) %>%
  dyRangeSelector()

```



```

#
=====
=====

# EXPERIMENT 30

# Intrusion Detection and Anomaly Detection Simulation

#
=====
=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic intrusion detection data

days <- as.Date("2026-10-23") + 0:9

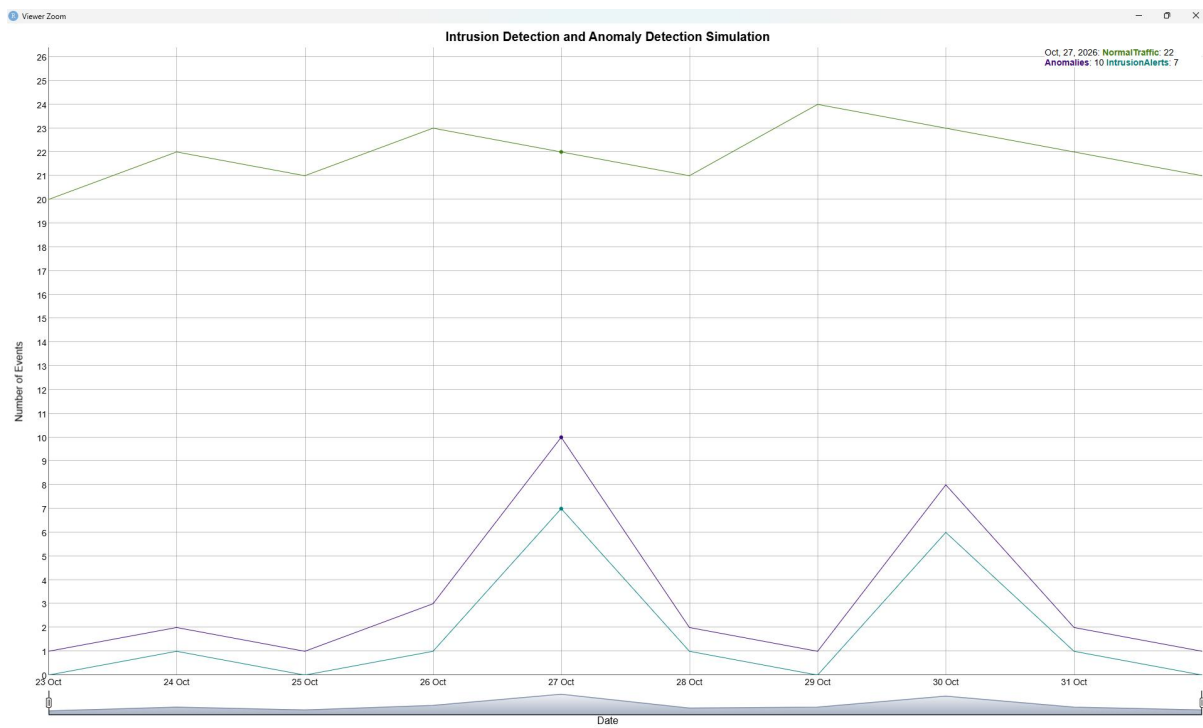

intrusion_data <- xts(
  cbind(
    NormalTraffic = c(20, 22, 21, 23, 22, 21, 24, 23, 22, 21),
    Anomalies = c(1, 2, 1, 3, 10, 2, 1, 8, 2, 1),
    IntrusionAlerts = c(0, 1, 0, 1, 7, 1, 0, 6, 1, 0)
  ),
  order.by = days
)

```

```

dygraph(
  intrusion_data,
  main = "Intrusion Detection and Anomaly Detection Simulation",
  xlab = "Date",
  ylab = "Number of Events"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 31

# Security Incident Classification and Severity Analysis

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic incident severity data

days <- as.Date("2026-10-26") + 0:9

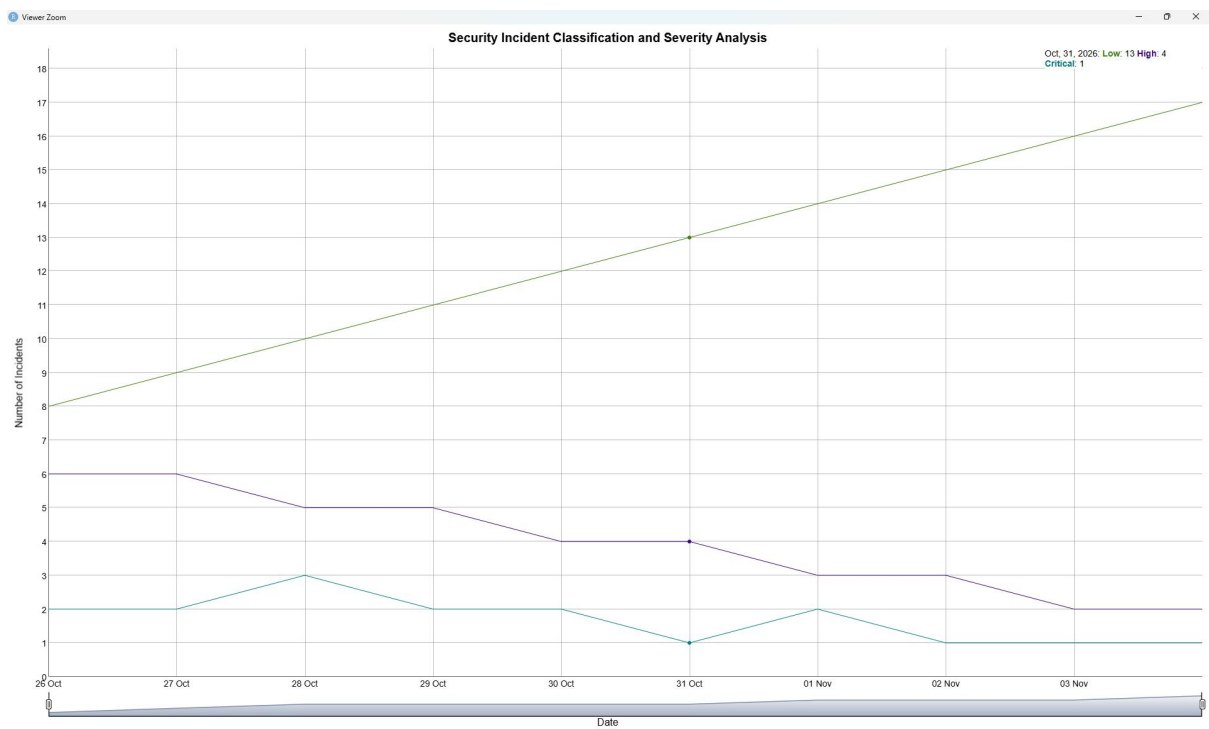

incident_data <- xts(
  cbind(
    Low = c(8, 9, 10, 11, 12, 13, 14, 15, 16, 17),
    High = c(6, 6, 5, 5, 4, 4, 3, 3, 2, 2),
    Critical = c(2, 2, 3, 2, 2, 1, 2, 1, 1, 1)
  ),
  order.by = days
)

```

```

dygraph(
  incident_data,
  main = "Security Incident Classification and Severity Analysis",
  xlab = "Date",
  ylab = "Number of Incidents"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 32

# Incident Response Workflow Modelling and Simulation

#
=====

=====

# Run once if required

install.packages("DiagrammeR")

library(DiagrammeR)

grViz("
digraph IncidentResponse {

graph [
  rankdir=LR,
  bgcolor=white,
  label='Incident Response Workflow',
  labelloc=t,
  fontsize=20
]

node [
  shape=box,

```



```
style='rounded, filled',  
fontname=Helvetica,  
fontsize=13  
]
```

```
Detect [  
    label='Detect\\nSecurity Incident',  
    fillcolor=lightblue  
]
```

```
Classify [  
    label='Classify\\nIncident',  
    fillcolor=lightyellow  
]
```

```
Contain [  
    label='Contain\\nThreat',  
    fillcolor=lightpink  
]
```

```
Eradicate [  
    label='Eradicate\\nRoot Cause',  
    fillcolor=lightgreen  
]
```

```
Recover [  
]
```

```
label='Recover\\nSystem',  
fillcolor=lightcyan  
]
```

```
Monitor [  
label='Monitor\\n& Verify',  
fillcolor=lavender  
]
```

```
Report [  
label='Incident Report\\n& Lessons Learned',  
fillcolor=wheat  
]
```

Detect -> Classify

Classify -> Contain

Contain -> Eradicate

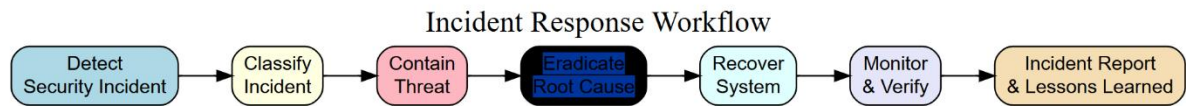
Eradicate -> Recover

Recover -> Monitor

Monitor -> Report

```
}
```

```
")
```



```
#
=====
```

```
# EXPERIMENT 33
```

```
# Security Incident Timeline Reconstruction and Root Cause Analysis
```

```
#
=====
```

```
# Run once if required
```

```
install.packages("dygraphs")
```

```
install.packages("xts")
```

```
library(dygraphs)
```

```
library(xts)
```

```

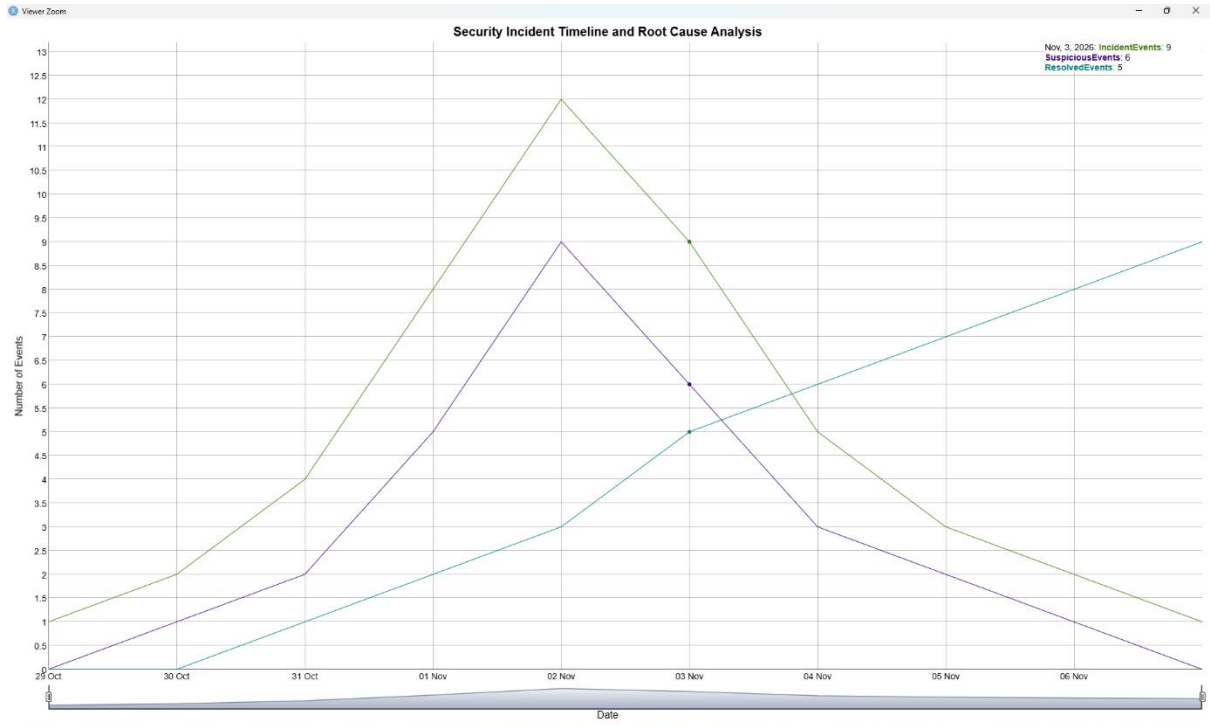
# Synthetic incident timeline data

days <- as.Date("2026-10-29") + 0:9

incident_data <- xts(
  cbind(
    IncidentEvents = c(1, 2, 4, 8, 12, 9, 5, 3, 2, 1),
    SuspiciousEvents = c(0, 1, 2, 5, 9, 6, 3, 2, 1, 0),
    ResolvedEvents = c(0, 0, 1, 2, 3, 5, 6, 7, 8, 9)
  ),
  order.by = days
)

dygraph(
  incident_data,
  main = "Security Incident Timeline and Root Cause Analysis",
  xlab = "Date",
  ylab = "Number of Events"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 34

# MTTD and MTTR Analysis

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic MTTD and MTTR data

days <- as.Date("2026-11-02") + 0:9

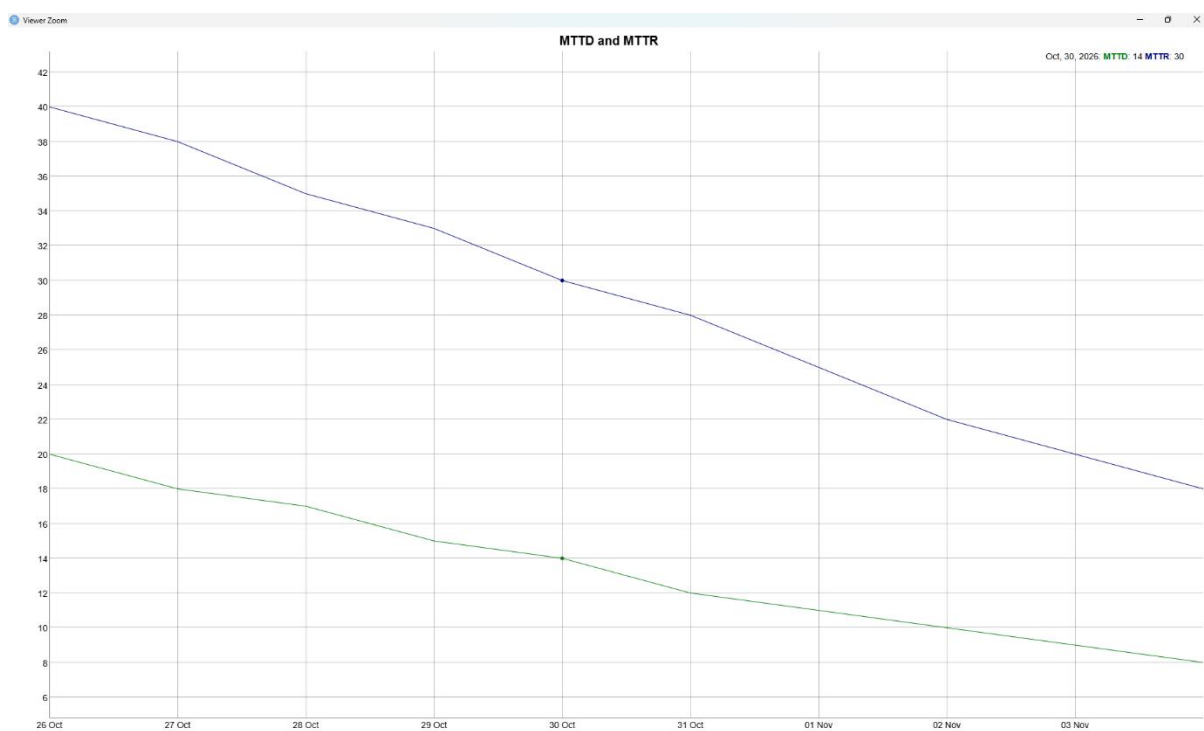

incident_data <- xts(
  cbind(
    MTTD = c(20, 18, 17, 15, 14, 12, 11, 10, 9, 8),
    MTTR = c(40, 38, 35, 33, 30, 28, 25, 22, 20, 18)
  ),
  order.by = days
)

```

```

dygraph(
  incident_data,
  main = "MTTD and MTTR Analysis",
  xlab = "Date",
  ylab = "Time (Minutes)"
) %>%
  dyRangeSelector()

```



```

#
=====

=====

# EXPERIMENT 35

# Security Compliance Monitoring and Governance Simulation

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic security compliance data

days <- as.Date("2026-11-05") + 0:9


compliance_data <- xts(
  cbind(
    ComplianceScore = c(70, 72, 75, 77, 80, 82, 84, 87, 90, 92),
    PoliciesSatisfied = c(65, 68, 70, 73, 76, 79, 82, 85, 88, 91),
    AuditFindings = c(15, 13, 12, 10, 9, 8, 6, 5, 3, 2)
  ),
  order.by = days
)

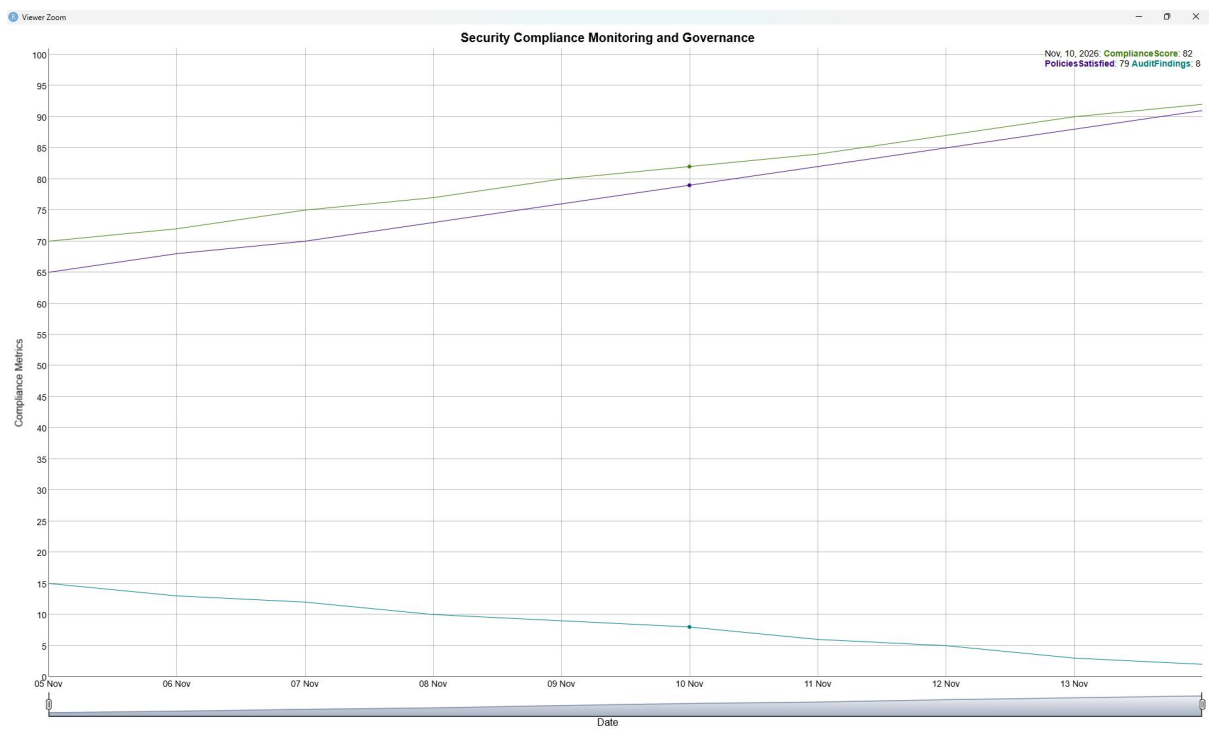
```



```

dygraph(
  compliance_data,
  main = "Security Compliance Monitoring and Governance",
  xlab = "Date",
  ylab = "Compliance Metrics"
) %>%
  dyRangeSelector()

```



```

#
=====
=====

# EXPERIMENT 36

# DevSecOps Risk-Based Decision Making using Synthetic Data

#
=====
=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic risk-based decision data

days <- as.Date("2026-11-07") + 0:9

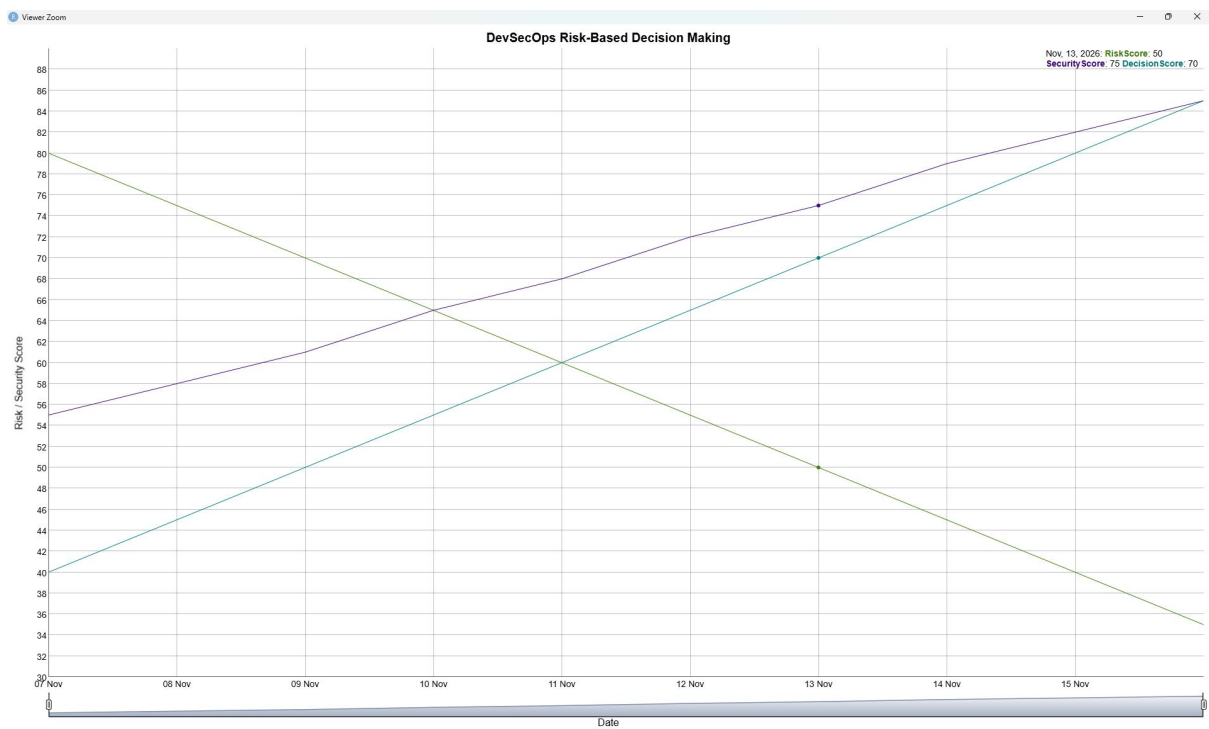

risk_data <- xts(
  cbind(
    RiskScore = c(80, 75, 70, 65, 60, 55, 50, 45, 40, 35),
    SecurityScore = c(55, 58, 61, 65, 68, 72, 75, 79, 82, 85),
    DecisionScore = c(40, 45, 50, 55, 60, 65, 70, 75, 80, 85)
  ),
  order.by = days
)

```

```

dygraph(
  risk_data,
  main = "DevSecOps Risk-Based Decision Making",
  xlab = "Date",
  ylab = "Risk / Security Score"
) %>%
  dyRangeSelector()

```



```
#
=====

=====

# EXPERIMENT 37

# Monte Carlo Simulation for DevSecOps Security Risk Analysis

#
=====

=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Generate Monte Carlo risk simulation data

set.seed(1)


simulations <- 100

risk <- rnorm(simulations, mean = 50, sd = 10)


# Keep risk values within a practical range

risk <- pmax(0, pmin(100, risk))

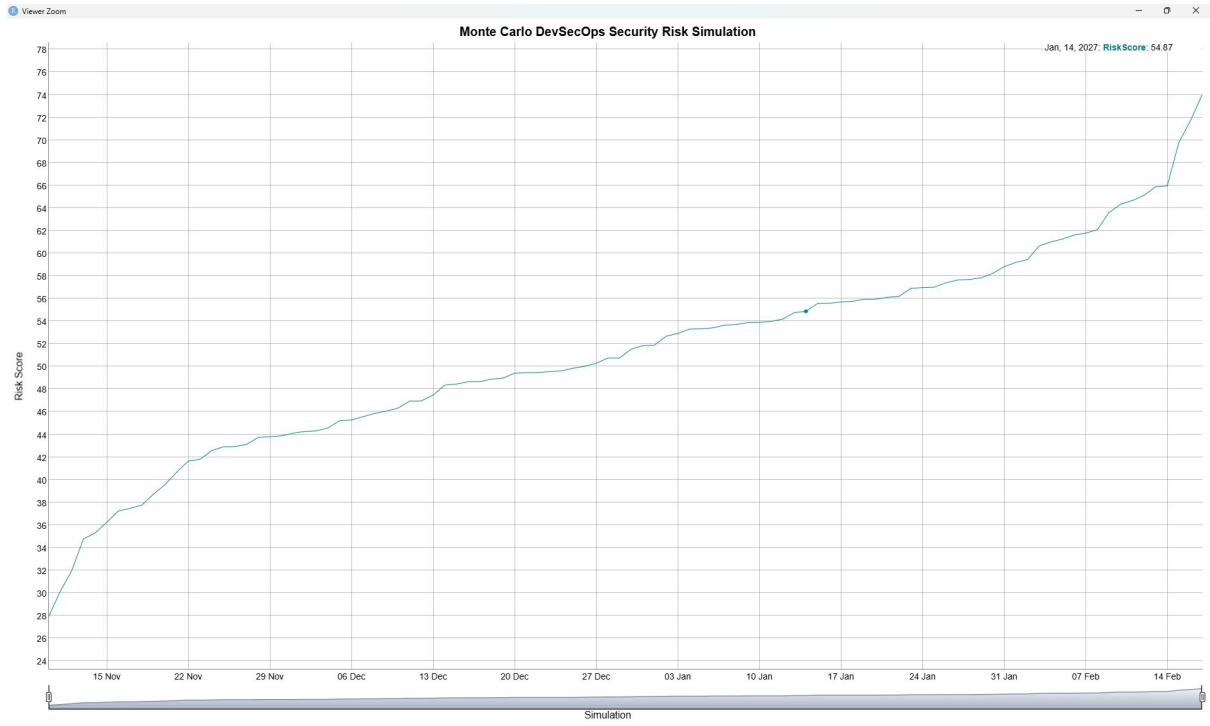

# Sort risk values for clear visualization

risk <- sort(risk)
```

```
days <- as.Date("2026-11-10") + 0:(simulations - 1)
```

```
risk_data <- xts(  
  cbind(  
    RiskScore = risk  
  ),  
  order.by = days  
)
```

```
dygraph(  
  risk_data,  
  main = "Monte Carlo DevSecOps Security Risk Simulation",  
  xlab = "Simulation",  
  ylab = "Risk Score"  
) %>%  
  dyRangeSelector()
```



```

#
=====
=====

# EXPERIMENT 38

# Comparative Evaluation of Secure and Insecure CI/CD Pipelines

#
=====
=====


# Run once if required

install.packages("dygraphs")
install.packages("xts")


library(dygraphs)
library(xts)


# Synthetic CI/CD pipeline comparison data
days <- as.Date("2026-11-12") + 0:9

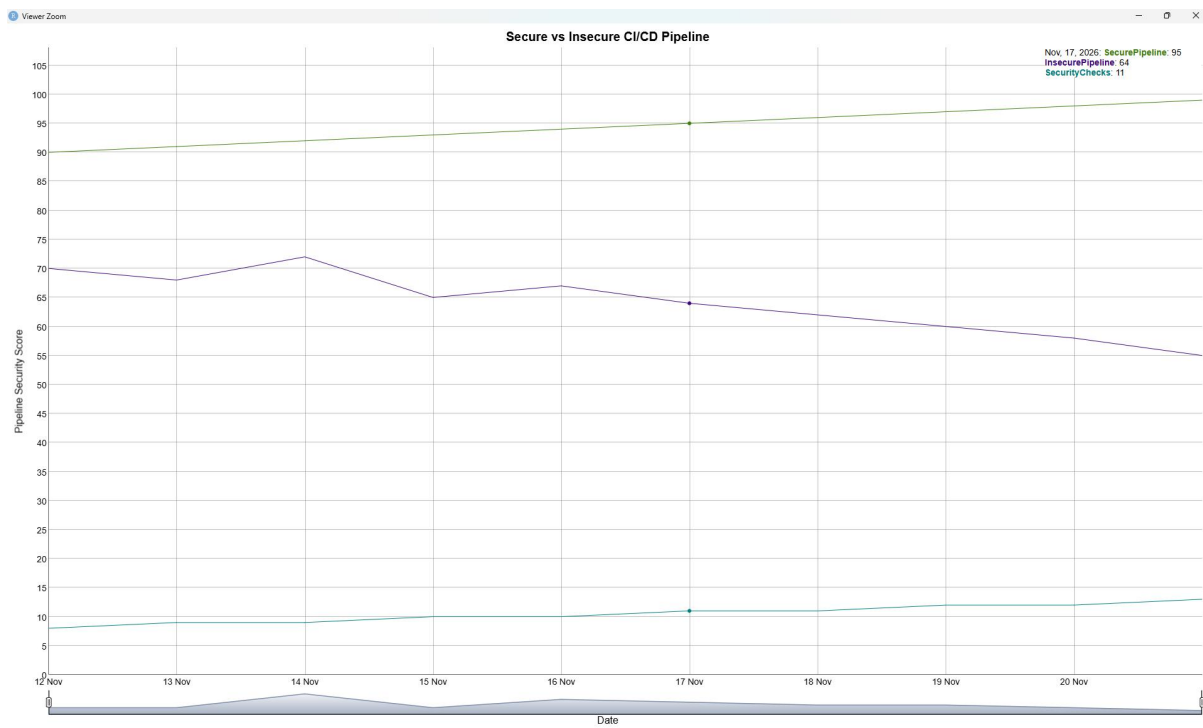

pipeline_data <- xts(
  cbind(
    SecurePipeline = c(90, 91, 92, 93, 94, 95, 96, 97, 98, 99),
    InsecurePipeline = c(70, 68, 72, 65, 67, 64, 62, 60, 58, 55),
    SecurityChecks = c(8, 9, 9, 10, 10, 11, 11, 12, 12, 13)
  ),
  order.by = days
)

```

```

dygraph(
  pipeline_data,
  main = "Secure vs Insecure CI/CD Pipeline",
  xlab = "Date",
  ylab = "Pipeline Security Score"
) %>%
  dyRangeSelector()

```




```

#
=====
=====

# EXPERIMENT 39

# Integrated DevSecOps Security Analytics and Reporting

#
=====
=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic integrated security analytics data

days <- as.Date("2026-11-14") + 0:9

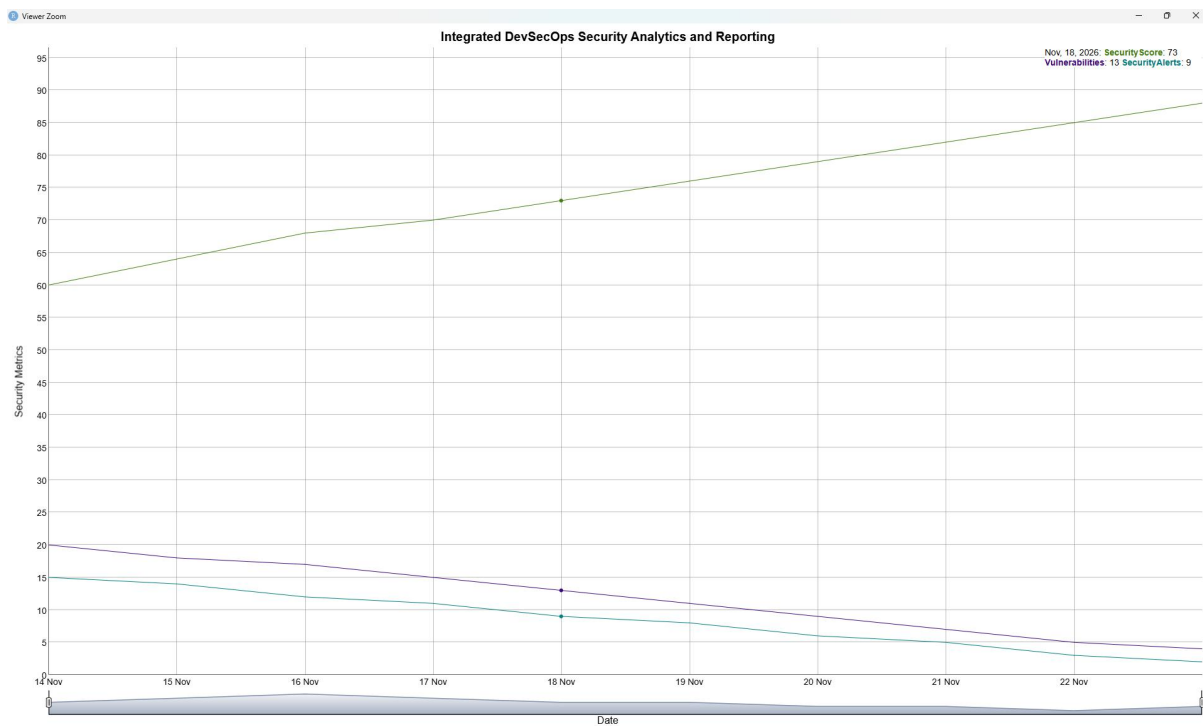

security_data <- xts(
  cbind(
    SecurityScore = c(60, 64, 68, 70, 73, 76, 79, 82, 85, 88),
    Vulnerabilities = c(20, 18, 17, 15, 13, 11, 9, 7, 5, 4),
    SecurityAlerts = c(15, 14, 12, 11, 9, 8, 6, 5, 3, 2)
  ),
  order.by = days
)

```

```

dygraph(
  security_data,
  main = "Integrated DevSecOps Security Analytics and Reporting",
  xlab = "Date",
  ylab = "Security Metrics"
) %>%
  dyRangeSelector()

```



```

#
=====
=====

# EXPERIMENT 40

# End-to-End DevSecOps Pipeline Security Simulation and Evaluation

#
=====
=====


# Run once if required

install.packages("dygraphs")

install.packages("xts")


library(dygraphs)

library(xts)


# Synthetic end-to-end DevSecOps pipeline data

days <- as.Date("2026-11-16") + 0:9


pipeline_data <- xts(
  cbind(
    Build = c(70, 72, 75, 78, 80, 82, 84, 86, 88, 90),
    Security = c(60, 64, 68, 72, 75, 78, 81, 84, 87, 90),
    Deployment = c(55, 60, 65, 68, 72, 76, 80, 84, 87, 91)
  ),
  order.by = days
)

```

```

dygraph(
  pipeline_data,
  main = "End-to-End DevSecOps Pipeline Security Simulation",
  xlab = "Date",
  ylab = "Pipeline Score"
) %>%
  dyRangeSelector()

```

